

SOLIDWORKS API (.NET VBA C#) — Examples (16/50): Create_Sketch_Point_Example_VB.NET- Create_Swept_Flange_Using_Gauge_Table_Ex

SOLIDWORKS API Documentation (converted from CHM)

Create a Sketch Point Example (VB.NET)

This example shows how to create a sketch point.

 ' Preconditions: Verify that the specified part template exists.

' Postconditions:
 ' 1. Opens a new part document and creates a sketch.
 ' 2. Creates a sketch point in the sketch.
 ' 3. Examine the graphics area.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swSkMgr As SketchManager
        Dim longstatus As Integer
        Dim boolstatus As Boolean
```

```
        swApp.ResetUntitledCount(0, 0, 0)
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2012\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        swModel = swApp.ActiveDoc
```

```
        swSkMgr = swModel.SketchManager
        swSkMgr.InsertSketch(True)
        boolstatus = swModel.Extension.SelectByID("Top Plane", "PLANE", -0.0553489443349025, 0.00330468607538553,
0.0269617286188933, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
```

```
        ' Check whether document is active
```

```
        If swModel Is Nothing Then
            swApp.SendMsgToUser2("A part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)
            Exit Sub
        End If
```

```
        ' Check whether document is a part
```

```
        Dim modelType As Integer
        modelType = swModel.GetType
```

```
        If modelType <> swDocumentTypes_e.swDocPART Then
            swApp.SendMsgToUser2("A part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)
            Exit Sub
        End If
```

```
        Dim skPoint As SketchPoint
        skPoint = swSkMgr.CreatePoint(-0.127443, 0.042892, 0.0#)
        swSkMgr.InsertSketch(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create Slot Mate Example (C#)

This example shows how to create a slot mechanical mate.

```
// -----
// Preconditions: Open
// public_documents\samples\tutorial\api\MechanicalMates\slot_slot.sldasm.
//
// Postconditions: Inspect the Mates folder of the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateSlotMate_CSharp
{
    partial class SolidWorksMacro
    {

        private ModelDoc2 Part;
        private AssemblyDoc swAssembly;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)Part;
            boolstatus = Part.Extension.SelectByRay(0.12649032355856, 0.133857958976421, 0.00879467058769023, 0.0353905007657348, 0.
            boolstatus = Part.Extension.SelectByRay(0.0959257342875013, 0.0999222046038994, -0.0251429018803719, 0.0353905007657348,

            // Create SlotMateFeatureData
            SlotMateFeatureData MateData = (SlotMateFeatureData)swAssembly.CreateMateData(21);

            // Set the entities to mate
            object[] EntitiesToMate = new object[2];
            EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
            object EntitiesToMateVar = EntitiesToMate;
            MateData.EntitiesToMate = (EntitiesToMateVar);

            // Set the mate alignment
            MateData.MateAlignment = 0;

            // Set the mate constraint
            MateData.Constraint = 0;

            // Create the mate
            Feature MateFeature = (Feature)swAssembly.CreateMate(MateData);
            Part.ClearSelection2(true);
            Part.EditRebuild3();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Sheet Metal Corner Relief Feature Example (C#)

This example shows how to create sheet metal corner reliefs.

```
// =====
// Preconditions:
// 1. Ensure that the specified part template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Press F5.
// 2. Creates:
//    - Base-Flange1
//    - Sketched Bend1
//    - Sketched Bend2
//    - Sketched Bend3
//    - Sketched Bend4
// 3. Selects the sheet metal solid body.
// 4. Sets rectangular reliefs for the four corners.
// 5. Creates Corner Relief1.
// 6. Press F5.
// 7. Sets circular reliefs for the four corners.
// 8. Press F5.
// 9. Sets constant width reliefs for the four corners.
// 10. Inspect the Immediate window and the graphics area after each change.
// =====
```

```
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace Macro1_CSharp
```

```
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private FeatureManager featMgr;
        private CornerReliefFeatureData cornerReliefFeatData;
        private Body2 swBody;
        private SelectionMgr selMgr;
        private object[] getCornerVar;
        private object corners;
        private int cntr1;
        private SMCornerReliefData cornerItr;
        private Feature Feat;
        private CornerReliefFeatureData featModData;
        private CornerReliefFeatureData cornerReliefMod3Data;
        private bool UseRatio;
        private object CFace1;
        private object CFace2;
        private object CFace3;
        private bool boolstatus;
        private int longstatus;

        public void PrintReliefData(int cornerIndex, CornerReliefFeatureData cornerReliefFeatData)
        {
            object smCornerReliefData1;
            longstatus = cornerReliefFeatData.GetCornerAtIndex(cornerIndex, out smCornerReliefData1);
        }
    }
}
```

```

        Debug.Print("Add filleted corners? " + ((SMCornerReliefData)smCornerReliefData1).AddFilletedCorners);
        Debug.Print("Corner fillet diameter = " + ((SMCornerReliefData)smCornerReliefData1).CornerFilletDiameter);
        Debug.Print("Use ratio to thickness? " + ((SMCornerReliefData)smCornerReliefData1).RatioToThickness);
        Debug.Print("Relief type as defined by swCornerBendReliefType_e = " +
((SMCornerReliefData)smCornerReliefData1).ReliefType);
        Debug.Print("Slot length = " + ((SMCornerReliefData)smCornerReliefData1).SlotLength);
        Debug.Print("Slot width = " + ((SMCornerReliefData)smCornerReliefData1).SlotWidth);
        Debug.Print("Center on bend lines? " + ((SMCornerReliefData)smCornerReliefData1).CenterOnBendLines);
        Debug.Print("Tangent to bend? " + ((SMCornerReliefData)smCornerReliefData1).TangentToBend);
        ((SMCornerReliefData)smCornerReliefData1).GetFaces(out CFace1, out CFace2, out CFace3);
    }

    public void Main()
    {
        swModel = (ModelDoc2)swApp.ActiveDoc;
        double swSheetWidth;
        swSheetWidth = 0d;
        double swSheetHeight;
        swSheetHeight = 0d;
        swModel = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0,
swSheetWidth, swSheetHeight);
        featMgr = swModel.FeatureManager;
        boolstatus = swModel.Extension.SelectByID("Front Plane", "PLANE", -0.0754055245830957d, 0.0542345258765876d,
0.00294433115750138d, false, 0, null, 0);
        swModel.SketchManager.InsertSketch(true);
        swModel.ClearSelection2(true);
        object skSegment;
        skSegment = swModel.SketchManager.CreateLine(-0.049594d, 0.029143d, 0d, 0.026075d, 0.029143d, 0d);
        skSegment = swModel.SketchManager.CreateLine(0.026075d, 0.029143d, 0d, 0.026075d, 0.015645d, 0d);
        skSegment = swModel.SketchManager.CreateLine(0.026075d, 0.015645d, 0d, 0.04305d, 0.015645d, 0d);
        skSegment = swModel.SketchManager.CreateLine(0.04305d, 0.015645d, 0d, 0.04305d, -0.034869d, 0d);
        skSegment = swModel.SketchManager.CreateLine(0.04305d, -0.034869d, 0d, 0.026075d, -0.034869d, 0d);
        skSegment = swModel.SketchManager.CreateLine(0.026075d, -0.034869d, 0d, 0.026075d, -0.048571d, 0d);
        skSegment = swModel.SketchManager.CreateLine(0.026075d, -0.048571d, 0d, -0.049594d, -0.048571d, 0d);
        skSegment = swModel.SketchManager.CreateLine(-0.049594d, -0.048571d, 0d, -0.049594d, -0.034869d, 0d);
        skSegment = swModel.SketchManager.CreateLine(-0.049594d, -0.034869d, 0d, -0.066568d, -0.034869d, 0d);
        skSegment = swModel.SketchManager.CreateLine(-0.066568d, -0.034869d, 0d, -0.066568d, 0.015645d, 0d);
        skSegment = swModel.SketchManager.CreateLine(-0.066568d, 0.015645d, 0d, -0.049594d, 0.015645d, 0d);
        skSegment = swModel.SketchManager.CreateLine(-0.049594d, 0.015645d, 0d, -0.049594d, 0.029143d, 0d);
        swModel.ClearSelection2(true);
        swModel.SketchManager.InsertSketch(true);
        swModel.ShowNamedView2("**Trimetric", 8);
        swModel.ViewZoomtofit2();
        swModel.SketchManager.InsertSketch(true);
        CustomBendAllowance customBendAllowanceData;
        customBendAllowanceData = swModel.FeatureManager.CreateCustomBendAllowance();
        customBendAllowanceData.KFactor = 0.5d;
        Feature myFeature;
        myFeature = swModel.FeatureManager.InsertSheetMetalBaseFlange2(0.0007366d, false, 0.0007366d, 0.02d, 0.01d, false, 0,
0, 1, customBendAllowanceData, false, 0, 0.0001d, 0.0001d, 0.5d, true, false, true, true);
        boolstatus = swModel.Extension.SelectByRay(-0.0145172925108454d, -0.00387700058865903d, 0, -0.5d, -0.707106781186547d,
-0.5d, 0.000556802642645787d, 2, false, 0, 0);
        swModel.SketchManager.InsertSketch(true);
        swModel.ClearSelection2(true);
        skSegment = swModel.SketchManager.CreateLine(-0.049594d, 0.015645d, 0d, -0.049594d, -0.034869d, 0d);
        swModel.ClearSelection2(true);
        swModel.SketchManager.InsertSketch(true);
        swModel.SketchManager.InsertSketch(true);
        boolstatus = swModel.Extension.SelectByRay(-0.0254254889823514d, -0.00675397704068888d, 0, 0, 0, -1,
0.000428258640921609d, 2, true, 0, 0);
        var CBAObject = default(CustomBendAllowance);
        myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949d, true, 0.0007366d, false, 3, CBAObject);
        boolstatus = swModel.Extension.SelectByRay(-0.0246697384395485d, -0.00297522432667467d, 0, 0, 0, -1,
0.000428258640921609d, 2, false, 0, 0);
    }

```

```

swModel.SketchManager.InsertSketch(true);
swModel.ClearSelection2(true);
boolstatus = swModel.Extension.SelectByRay(-0.049594d, 0.015645d, 0, 0, 0, -1, 0.000392913800359539d, 3, false, 0, 0);
swModel.ClearSelection2(true);
skSegment = swModel.SketchManager.CreateLine(-0.049594d, 0.015645d, 0d, 0.026075d, 0.015645d, 0d);
swModel.ClearSelection2(true);
swModel.SketchManager.InsertSketch(true);
swModel.SketchManager.InsertSketch(true);
boolstatus = swModel.Extension.SelectByRay(-0.0235965085767476d, -0.00434032596567101d, 0, 0, 0, -1,
0.000392913800359539d, 2, true, 0, 0);
myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949d, true, 0.0007366d, false, 3, CBAObject);
boolstatus = swModel.Extension.SelectByRay(-0.0234809456942889d, -0.0197101893326765d, 0, 0, 0, -1,
0.000392913800359539d, 2, false, 0, 0);
swModel.SketchManager.InsertSketch(true);
swModel.ClearSelection2(true);
skSegment = swModel.SketchManager.CreateLine(-0.049594d, -0.034869d, 0d, 0.026075d, -0.034869d, 0d);
swModel.ClearSelection2(true);
swModel.SketchManager.InsertSketch(true);
swModel.SketchManager.InsertSketch(true);
boolstatus = swModel.Extension.SelectByRay(-0.0171484564194607d, -0.00399245414393841d, 0, 0, 0, -1,
0.000367699685812068d, 2, true, 0, 0);
myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949d, true, 0.0007366d, false, 3, CBAObject);
boolstatus = swModel.Extension.SelectByRay(-0.0107677854009571d, -0.00972424336395007d, 0, 0, 0, -1,
0.000367699685812068d, 2, false, 0, 0);
swModel.SketchManager.InsertSketch(true);
swModel.ClearSelection2(true);
skSegment = swModel.SketchManager.CreateLine(0.026075d, -0.034869d, 0d, 0.026075d, 0.015645d, 0d);
swModel.ClearSelection2(true);
swModel.SketchManager.InsertSketch(true);
swModel.SketchManager.InsertSketch(true);
boolstatus = swModel.Extension.SelectByRay(-0.0259083606991011d, -0.0062053705579176d, 0, 0, 0, -1,
0.000367699685812068d, 2, true, 0, 0);
myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949d, true, 0.0007366d, false, 3, CBAObject);
// Zoom to Area
swModel.ViewZoomTo2(0.0152447662706814d, 0.00218199896815251d, -0.00886660359613402d, 0.022429935049572d,
-0.00638493303744786d, -0.00886660359613402d);
cornerReliefFeatData = (CornerReliefFeatureData)featMgr.CreateDefinition((int)swFeatureNameID_e.swFmCornerRelief);
boolstatus = swModel.Extension.SelectByID2("Sketched Bend4", "SOLIDBODY", 0, 0, 0, false, 0, null, 0);
selMgr = (SelectionMgr)swModel.SelectionManager;
swBody = (Body2)selMgr.GetSelectedObject6(1, -1);
cornerReliefFeatData.Initialize((int)swCornerReliefBendType_e.swCornerReliefBendType_TwoBend);
cornerReliefFeatData.SetBodyScope(swBody);

// Default is tear relief for all Sketched Bends
longstatus = cornerReliefFeatData.CollectAllCorners((int)swCornerReliefType_e.swCornerTearRelief, out corners);
getCornerVar = (object[])corners;
Debug.Print("");
Debug.Print("-----Creating corners with rectangular reliefs----- ");
var loopTo = getCornerVar.GetUpperBound(0);
for (cntr1 = 0; cntr1 <= loopTo; cntr1++)
{
    cornerItr = (SMCornerReliefData)getCornerVar[cntr1];
    cornerItr.AddFilletedCorners = true;
    cornerItr.CornerFilletDiameter = 0.00025d;
    cornerItr.CenterOnBendLines = true;
    cornerItr.TangentToBend = true;
    cornerItr.ReliefType = (int)swCornerReliefType_e.swCornerRectangularRelief;
    UseRatio = true;
    if (!UseRatio)
    {
        cornerItr.SlotLength = 0.009d;
    }
    else
    {

```

```

        cornerItr.RatioToThickness = true;
    }

    PrintReliefData(cnr1 + 1, cornerReliefFeatData);
    Debug.Print("");
}

Feat = featMgr.CreateFeature(cornerReliefFeatData);
Debugger.Break();
featModData = (CornerReliefFeatureData)Feat.GetDefinition();
featModData.AccessSelections(swModel, null);
getCornerVar = (object[])featModData.GetCorners(-1);
Debug.Print("");
Debug.Print("-----Changing corners to circular reliefs----- ");
var loopTo1 = getCornerVar.GetUpperBound(0);
for (cnr1 = 0; cnr1 <= loopTo1; cnr1++)
{
    cornerItr = (SMCornerReliefData)getCornerVar[cnr1];
    cornerItr.ReliefType = (int)swCornerReliefType_e.swCornerCircularRelief;
    UseRatio = false;
    if (!UseRatio)
    {
        cornerItr.SlotWidth = 0.0008d;
    }
    else
    {
        cornerItr.RatioToThickness = true;
    }

    PrintReliefData(cnr1 + 1, featModData);
    Debug.Print("");
}

Feat.ModifyDefinition(featModData, swModel, null);
Debugger.Break();
cornerReliefMod3Data = (CornerReliefFeatureData)Feat.GetDefinition();
cornerReliefMod3Data.AccessSelections(swModel, null);
getCornerVar = (object[])cornerReliefMod3Data.GetCorners(-1);
Debug.Print("");
Debug.Print("-----Changing corners to constant width reliefs----- ");
var loopTo2 = getCornerVar.GetUpperBound(0);
for (cnr1 = 0; cnr1 <= loopTo2; cnr1++)
{
    cornerItr = (SMCornerReliefData)getCornerVar[cnr1];
    cornerItr.ReliefType = (int)swCornerReliefType_e.swCornerConstantWidthRelief;
    PrintReliefData(cnr1 + 1, cornerReliefMod3Data);
    Debug.Print("");
}

Feat.ModifyDefinition(cornerReliefMod3Data, swModel, null);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create Sheet Metal Corner Relief Feature Example (VB.NET)

This example shows how to create sheet metal corner reliefs.

```
'=====
'Preconditions:
'1. Ensure that the specified part template exists.
'2. Open an Immediate window.
'
'Postconditions:
' 1. Press F5.
' 2. Creates:
'    - Base-Flange1
'    - Sketched Bend1
'    - Sketched Bend2
'    - Sketched Bend3
'    - Sketched Bend4
' 3. Selects the sheet metal solid body.
' 4. Sets rectangular reliefs for the four corners.
' 5. Creates Corner Relief1.
' 6. Press F5.
' 7. Sets circular reliefs for the four corners.
' 8. Press F5.
' 9. Sets constant width reliefs for the four corners.
'10. Inspect the Immediate window and the graphics area after each change.
'=====

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2
    Dim featMgr As FeatureManager
    Dim cornerReliefFeatData As CornerReliefFeatureData
    Dim swBody As Body2
    Dim selMgr As SelectionMgr
    Dim getCornerVar() As Object
    Dim cntr1 As Integer
    Dim cornerItr As SMCornerReliefData
    Dim Feat As Feature
    Dim featModData As CornerReliefFeatureData
    Dim cornerReliefMod3Data As CornerReliefFeatureData
    Dim UseRatio As Boolean
    Dim CFace1 As Face2
    Dim CFace2 As Face2
    Dim CFace3 As Face2
    Dim boolstatus As Boolean
    Dim longstatus As Integer

    Sub PrintReliefData(ByVal cornerIndex As Integer, ByVal cornerReliefFeatData As CornerReliefFeatureData)

        Dim smCornerReliefData1 As SMCornerReliefData
        boolstatus = cornerReliefFeatData.GetCornerAtIndex(cornerIndex, smCornerReliefData1)

        Debug.Print("Add filleted corners? " & smCornerReliefData1.AddFilletedCorners)
        Debug.Print("Corner fillet diameter = " & smCornerReliefData1.CornerFilletDiameter)
```

```

Debug.Print("Use ratio to thickness? " & smCornerReliefData1.RatioToThickness)
Debug.Print("Relief type as defined by swCornerBendReliefType_e = " & smCornerReliefData1.ReliefType)
Debug.Print("Slot length = " & smCornerReliefData1.SlotLength)
Debug.Print("Slot width = " & smCornerReliefData1.SlotWidth)
Debug.Print("Center on bend lines? " & smCornerReliefData1.CenterOnBendLines)
Debug.Print("Tangent to bend? " & smCornerReliefData1.TangentToBend)

Call smCornerReliefData1.GetFaces(CFace1, CFace2, CFace3)

End Sub

Sub main()

    swModel = swApp.ActiveDoc

    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0, swSheetWidth,
swSheetHeight)
    featMgr = swModel.FeatureManager
    boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", -0.0754055245830957, 0.0542345258765876,
0.00294433115750138, False, 0, Nothing, 0)
    swModel.SketchManager.InsertSketch(True)
    swModel.ClearSelection2(True)
    Dim skSegment As Object
    skSegment = swModel.SketchManager.CreateLine(-0.049594, 0.029143, 0#, 0.026075, 0.029143, 0#)
    skSegment = swModel.SketchManager.CreateLine(0.026075, 0.029143, 0#, 0.026075, 0.015645, 0#)
    skSegment = swModel.SketchManager.CreateLine(0.026075, 0.015645, 0#, 0.04305, 0.015645, 0#)
    skSegment = swModel.SketchManager.CreateLine(0.04305, 0.015645, 0#, 0.04305, -0.034869, 0#)
    skSegment = swModel.SketchManager.CreateLine(0.04305, -0.034869, 0#, 0.026075, -0.034869, 0#)
    skSegment = swModel.SketchManager.CreateLine(0.026075, -0.034869, 0#, 0.026075, -0.048571, 0#)
    skSegment = swModel.SketchManager.CreateLine(0.026075, -0.048571, 0#, -0.049594, -0.048571, 0#)
    skSegment = swModel.SketchManager.CreateLine(-0.049594, -0.048571, 0#, -0.049594, -0.034869, 0#)
    skSegment = swModel.SketchManager.CreateLine(-0.049594, -0.034869, 0#, -0.066568, -0.034869, 0#)
    skSegment = swModel.SketchManager.CreateLine(-0.066568, -0.034869, 0#, -0.066568, 0.015645, 0#)
    skSegment = swModel.SketchManager.CreateLine(-0.066568, 0.015645, 0#, -0.049594, 0.015645, 0#)
    skSegment = swModel.SketchManager.CreateLine(-0.049594, 0.015645, 0#, -0.049594, 0.029143, 0#)
    swModel.ClearSelection2(True)
    swModel.SketchManager.InsertSketch(True)

    swModel.ShowNamedView2("**Trimetric", 8)
    swModel.ViewZoomtofit2
    swModel.SketchManager.InsertSketch(True)
    Dim customBendAllowanceData As CustomBendAllowance
    customBendAllowanceData = swModel.FeatureManager.CreateCustomBendAllowance()
    customBendAllowanceData.KFactor = 0.5
    Dim myFeature As Feature
    myFeature = swModel.FeatureManager.InsertSheetMetalBaseFlange2(0.0007366, False, 0.0007366, 0.02, 0.01, False, 0, 0, 1,
customBendAllowanceData, False, 0, 0.0001, 0.0001, 0.5, True, False, True, True)

    boolstatus = swModel.Extension.SelectByRay(-0.0145172925108454, -0.00387700058865903, 0, -0.5, -0.707106781186547, -0.5,
0.000556802642645787, 2, False, 0, 0)
    swModel.SketchManager.InsertSketch(True)
    swModel.ClearSelection2(True)
    skSegment = swModel.SketchManager.CreateLine(-0.049594, 0.015645, 0#, -0.049594, -0.034869, 0#)
    swModel.ClearSelection2(True)
    swModel.SketchManager.InsertSketch(True)
    swModel.SketchManager.InsertSketch(True)
    boolstatus = swModel.Extension.SelectByRay(-0.0254254889823514, -0.00675397704068888, 0, 0, 0, -1, 0.000428258640921609, 2,
True, 0, 0)
    Dim CBAObject As CustomBendAllowance

```

```

myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949, True, 0.0007366, False, 3, CBAObject)
boolstatus = swModel.Extension.SelectByRay(-0.0246697384395485, -0.00297522432667467, 0, 0, 0, -1, 0.000428258640921609, 2,
False, 0, 0)
swModel.SketchManager.InsertSketch(True)
swModel.ClearSelection2(True)
boolstatus = swModel.Extension.SelectByRay(-0.049594, 0.015645, 0, 0, 0, -1, 0.000392913800359539, 3, False, 0, 0)
swModel.ClearSelection2(True)
skSegment = swModel.SketchManager.CreateLine(-0.049594, 0.015645, 0#, 0.026075, 0.015645, 0#)
swModel.ClearSelection2(True)
swModel.SketchManager.InsertSketch(True)
swModel.SketchManager.InsertSketch(True)
boolstatus = swModel.Extension.SelectByRay(-0.0235965085767476, -0.00434032596567101, 0, 0, 0, -1, 0.000392913800359539, 2,
True, 0, 0)
myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949, True, 0.0007366, False, 3, CBAObject)
boolstatus = swModel.Extension.SelectByRay(-0.0234809456942889, -0.0197101893326765, 0, 0, 0, -1, 0.000392913800359539, 2,
False, 0, 0)
swModel.SketchManager.InsertSketch(True)
swModel.ClearSelection2(True)
skSegment = swModel.SketchManager.CreateLine(-0.049594, -0.034869, 0#, 0.026075, -0.034869, 0#)
swModel.ClearSelection2(True)
swModel.SketchManager.InsertSketch(True)
swModel.SketchManager.InsertSketch(True)
boolstatus = swModel.Extension.SelectByRay(-0.0171484564194607, -0.00399245414393841, 0, 0, 0, -1, 0.000367699685812068, 2,
True, 0, 0)
myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949, True, 0.0007366, False, 3, CBAObject)
boolstatus = swModel.Extension.SelectByRay(-0.0107677854009571, -0.00972424336395007, 0, 0, 0, -1, 0.000367699685812068, 2,
False, 0, 0)
swModel.SketchManager.InsertSketch(True)
swModel.ClearSelection2(True)
skSegment = swModel.SketchManager.CreateLine(0.026075, -0.034869, 0#, 0.026075, 0.015645, 0#)
swModel.ClearSelection2(True)
swModel.SketchManager.InsertSketch(True)
swModel.SketchManager.InsertSketch(True)
boolstatus = swModel.Extension.SelectByRay(-0.0259083606991011, -0.0062053705579176, 0, 0, 0, -1, 0.000367699685812068, 2,
True, 0, 0)
myFeature = swModel.FeatureManager.InsertSheetMetal3dBend(1.5707963267949, True, 0.0007366, False, 3, CBAObject)
' Zoom to Area
swModel.ViewZoomTo2(0.0152447662706814, 0.00218199896815251, -0.00886660359613402, 0.022429935049572, -0.00638493303744786,
-0.00886660359613402)

cornerReliefFeatData = featMgr.CreateDefinition(swFeatureNameID_e.swFmCornerRelief)

boolstatus = swModel.Extension.SelectByID2("Sketched Bend4", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)

selMgr = swModel.SelectionManager

swBody = selMgr.GetSelectedObject6(1, -1)

cornerReliefFeatData.Initialize(swCornerReliefBendType_e.swCornerReliefBendType_TwoBend)
cornerReliefFeatData.SetBodyScope(swBody)

'Default is tear relief for all Sketched Bends
longstatus = cornerReliefFeatData.CollectAllCorners(swCornerReliefType_e.swCornerTearRelief, getCornerVar)

Debug.Print("")
Debug.Print("-----Creating corners with rectangular reliefs----- ")

For cnt1 = 0 To UBound(getCornerVar)

    cornerItr = getCornerVar(cnt1)

    cornerItr.AddFilletCorners = True
    cornerItr.CornerFilletDiameter = 0.00025
    cornerItr.CenterOnBendLines = True

```

```

cornerItr.TangentToBend = True
cornerItr.ReliefType = swCornerReliefType_e.swCornerRectangularRelief

UseRatio = True

If Not UseRatio Then
    cornerItr.SlotLength = 0.009
Else
    cornerItr.RatioToThickness = True
End If

PrintReliefData(cntr1 + 1, cornerReliefFeatData)
Debug.Print("")
Next

Feat = featMgr.CreateFeature(cornerReliefFeatData)

Stop

featModData = Feat.GetDefinition()

featModData.AccessSelections(swModel, Nothing)

getCornerVar = featModData.GetCorners(-1)

Debug.Print("")
Debug.Print("-----Changing corners to circular reliefs----- ")

For cntr1 = 0 To UBound(getCornerVar)

    cornerItr = getCornerVar(cntr1)

    cornerItr.ReliefType = swCornerReliefType_e.swCornerCircularRelief

    UseRatio = False

    If Not UseRatio Then
        cornerItr.SlotWidth = 0.0008
    Else
        cornerItr.RatioToThickness = True
    End If

    PrintReliefData(cntr1 + 1, featModData)
    Debug.Print("")

Next

Feat.ModifyDefinition(featModData, swModel, Nothing)

Stop

cornerReliefMod3Data = Feat.GetDefinition()

cornerReliefMod3Data.AccessSelections(swModel, Nothing)

getCornerVar = cornerReliefMod3Data.GetCorners(-1)

Debug.Print("")
Debug.Print("-----Changing corners to constant width reliefs----- ")

For cntr1 = 0 To UBound(getCornerVar)

```

```

        cornerItr = getCornerVar(cntr1)

        cornerItr.ReliefType = swCornerReliefType_e.swCornerConstantWidthRelief

        PrintReliefData(cntr1 + 1, cornerReliefMod3Data)
        Debug.Print("")

    Next

    Feat.ModifyDefinition(cornerReliefMod3Data, swModel, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Solid Bodies Using Geometry and Topology Methods Example (C#)

This example shows how to create a solid body using SOLIDWORKS geometry and topology methods.

```
//-----
// Preconditions: Verify that the specified part
// document template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a part containing a single solid
//    body feature that is the union of a box and
//    a cone.
// 3. Examine the graphics area and FeatureManager
//    design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Modeler swModeler = default(Modeler);
            Feature swFeat = default(Feature);
            double[] nConeParam = new double[9];
            object vConeArr = null;
            Body2 swConeBody = default(Body2);
            double[] nBoxParam = new double[9];
            object vBoxArr = null;
            Body2 swBoxBody = default(Body2);
            object[] vNewBodyArr = null;
            object vNewBody = null;
            PartDoc swNewPart = default(PartDoc);
            Body2 swNewBody = default(Body2);
            FaultEntity swFaultEnt = default(FaultEntity);
            int nRetVal = 0;
            int nCount = 0;

            // Form cone
            // Face center
            nConeParam[0] = 0.0;
            nConeParam[1] = 0.1;
            nConeParam[2] = 0.0;
            // Axis
            nConeParam[3] = 0.0;
            nConeParam[4] = 0.0;
            nConeParam[5] = 1.0;
            // Base radius
            nConeParam[6] = 0.2;
            // Top radius
            nConeParam[7] = 0.1;
            // Height
            nConeParam[8] = 0.3;
            vConeArr = nConeParam;

            // Form box
            // Face center
            nBoxParam[0] = 0.0;
            nBoxParam[1] = 0.1;
            nBoxParam[2] = 0.2;
            // Axis
            nBoxParam[3] = 0.0;
            nBoxParam[4] = 0.0;
            nBoxParam[5] = 1.0;
            // Width
            nBoxParam[6] = 0.3;
            // Length
            nBoxParam[7] = 0.25;
            // Height
            nBoxParam[8] = 0.4;
            vBoxArr = nBoxParam;

            swModeler = (Modeler)swApp.GetModeler();
            swConeBody = (Body2)swModeler.CreateBodyFromCone((vConeArr));
            swBoxBody = (Body2)swModeler.CreateBodyFromBox((vBoxArr));
            swFaultEnt = (FaultEntity)swConeBody.Check3;
            nCount = swFaultEnt.Count;
            if (nCount != 0)
            {
                Debug.Print("Faulty cone!");
                return;
            }
            swFaultEnt = (FaultEntity)swBoxBody.Check3;
            nCount = swFaultEnt.Count;
            if (nCount != 0)
            {
                Debug.Print("Faulty box!");
                return;
            }
            vNewBodyArr = (object[])swConeBody.Operations2((int)swBodyOperationType_e.SwBODYADD, swBoxBody, out nRetVal);

            swNewPart = (PartDoc)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2014\\templates\\Part.prtdot", 0, 0, 0);
            foreach (object vNewBody_loopVariable in vNewBodyArr)
            {

```

```

        vNewBody = vNewBody_loopVariable;
        swNewBody = (Body2)vNewBody;
        // Create solid body feature
        swFeat = (Feature)swNewPart.CreateFeatureFromBody3(swNewBody, false, (int)swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck + (int)swCreateFeatureBodyOpts_e.swCreateFeatureBodySimplify);
    }

    swModel = (ModelDoc2)swNewPart;
    swModel.ViewZoomToFit2();
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create Solid Bodies Using Geometry and Topology Methods Example (VBA)

This example shows how to create a solid body using SOLIDWORKS geometry and topology methods.

```
'-----
' Preconditions: Verify that the specified part
' document template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a part containing a single solid
'    body feature that is the union of a box and
'    a cone.
' 3. Examine the graphics area and FeatureManager
'    design tree.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModeler As SldWorks.Modeler
    Dim swFeat As SldWorks.Feature
    Dim nConeParam(8) As Double
    Dim vConeArr As Variant
    Dim swConeBody As SldWorks.Body2
    Dim nBoxParam(8) As Double
    Dim vBoxArr As Variant
    Dim swBoxBody As SldWorks.Body2
    Dim vNewBodyArr As Variant
    Dim vNewBody As Variant
    Dim swNewPart As SldWorks.PartDoc
    Dim swNewBody As SldWorks.Body2
    Dim swFaultEnt As SldWorks.FaultEntity
    Dim nRetVal As Long
    Dim nCount As Long

    ' Form cone
    ' Face center
    nConeParam(0) = 0#
    nConeParam(1) = 0.1
    nConeParam(2) = 0#
    ' Axis
    nConeParam(3) = 0#
    nConeParam(4) = 0#
    nConeParam(5) = 1#
    ' Base radius
    nConeParam(6) = 0.2
    ' Top radius
    nConeParam(7) = 0.1
    ' Height
    nConeParam(8) = 0.3
    vConeArr = nConeParam

    ' Form box
    ' Face center
    nBoxParam(0) = 0#
    nBoxParam(1) = 0.1
    nBoxParam(2) = 0.2
    ' Axis
    nBoxParam(3) = 0#
    nBoxParam(4) = 0#
    nBoxParam(5) = 1#
    ' Width
    nBoxParam(6) = 0.3
    ' Length
    nBoxParam(7) = 0.25
    ' Height
    nBoxParam(8) = 0.4
    vBoxArr = nBoxParam

    Set swApp = CreateObject("SldWorks.Application")
    Set swModeler = swApp.GetModeler
    Set swConeBody = swModeler.CreateBodyFromCone((vConeArr))
    Set swBoxBody = swModeler.CreateBodyFromBox((vBoxArr))

    Set swFaultEnt = swConeBody.Check3
    nCount = swFaultEnt.Count
    If nCount <> 0 Then
        Debug.Print "Faulty cone!"
        Exit Sub
    End If

    Set swFaultEnt = swBoxBody.Check3
    nCount = swFaultEnt.Count
    If nCount <> 0 Then
        Debug.Print "Faulty box!"
        Exit Sub
    End If

    vNewBodyArr = swConeBody.Operations2(swBodyOperationType_e.SWBODYADD, swBoxBody, nRetVal)

    Set swNewPart = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)

    For Each vNewBody In vNewBodyArr
        Set swNewBody = vNewBody
    Next
```

```
        ' Create solid body feature
        Set swFeat = swNewPart.CreateFeatureFromBody3(swNewBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck +
swCreateFeatureBodyOpts_e.swCreateFeatureBodySimplify)
    Next

    Set swModel = swNewPart
    swModel.ViewZoomtofit2

End Sub
```

Create Solid Bodies Using Geometry and Topology Methods Example (VB.NET)

This example shows how to create a solid body using SOLIDWORKS geometry and topology methods.

```

-----
' Preconditions: Verify that the specified part
' document template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a part containing a single solid
'    body feature that is the union of a box and
'    a cone.
' 3. Examine the graphics area and FeatureManager
'    design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModeler As Modeler
        Dim swFeat As Feature
        Dim nConeParam(8) As Double
        Dim vConeArr As Object
        Dim swConeBody As Body2
        Dim nBoxParam(8) As Double
        Dim vBoxArr As Object
        Dim swBoxBody As Body2
        Dim vNewBodyArr As Object
        Dim vNewBody As Object
        Dim swNewPart As PartDoc
        Dim swNewBody As Body2
        Dim swFaultEnt As FaultEntity
        Dim nRetVal As Integer
        Dim nCount As Integer

        ' Form cone
        ' Face center
        nConeParam(0) = 0.0#
        nConeParam(1) = 0.1
        nConeParam(2) = 0.0#
        ' Axis
        nConeParam(3) = 0.0#
        nConeParam(4) = 0.0#
        nConeParam(5) = 1.0#
        ' Base radius
        nConeParam(6) = 0.2
        ' Top radius
        nConeParam(7) = 0.1
        ' Height
        nConeParam(8) = 0.3
        vConeArr = nConeParam

        ' Form box
        ' Face center
        nBoxParam(0) = 0.0#
        nBoxParam(1) = 0.1
        nBoxParam(2) = 0.2
        ' Axis
        nBoxParam(3) = 0.0#
        nBoxParam(4) = 0.0#
        nBoxParam(5) = 1.0#
        ' Width
        nBoxParam(6) = 0.3
        ' Length
        nBoxParam(7) = 0.25
        ' Height
        nBoxParam(8) = 0.4
        vBoxArr = nBoxParam

        swModeler = swApp.GetModeler
        swConeBody = swModeler.CreateBodyFromCone((vConeArr))
        swBoxBody = swModeler.CreateBodyFromBox((vBoxArr))
        swFaultEnt = swConeBody.Check3
        nCount = swFaultEnt.Count
        If nCount <> 0 Then
            Debug.Print("Faulty cone!")
            Exit Sub
        End If
        swFaultEnt = swBoxBody.Check3
        nCount = swFaultEnt.Count
        If nCount <> 0 Then
            Debug.Print("Faulty box!")
            Exit Sub
        End If
        vNewBodyArr = swConeBody.Operations2(swBodyOperationType_e.SWBODYADD, swBoxBody, nRetVal)

        swNewPart = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2014\templates\Part.prtdot", 0, 0, 0)
        For Each vNewBody In vNewBodyArr
            swNewBody = vNewBody
            ' Create solid body feature
            swFeat = swNewPart.CreateFeatureFromBody3(swNewBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck + swCreateFeatureBodyOpts_e.swC
            reateFeatureBodySimplify)
        Next

        swModel = swNewPart
        swModel.ViewZoomtofit2()

    End Sub

''' <summary>
''' The Sldworks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As SldWorks

End Class

Create Solid Body Surface Trim Feature Example (C#)

This example shows how to create a solid body surface trim feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Creates Surface-Trim1.
// 3. Expand and examine Solid Bodies(1) in the FeatureManager
//    design tree and examine the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            bool status = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\SurfaceTrimFeature.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;

            // Select surface features
            status = swModelDocExt.SelectByID2("", "SURFACEBODY", -0.0446486526100784, 0.0218350174377093, 0.0123754341749418, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "SURFACEBODY", -0.00815686270678384, 0.0415839719953865, 0.0242402652081068, true, 0, null, 0);

            // Select trimming surfaces to create solid body surface trim feature
            status = swFeatureManager.PreTrimSurface(true, true, false, true);
            status = swModelDocExt.SelectByID2("", "SURFACEBODY", 0.0059504253577245, 0.0413800871671199, 0.0248740287174201, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "SURFACEBODY", -0.037205042299604, 0.0343527327176432, 0.0123446167727934, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "SURFACEBODY", -0.0104497983190015, -0.0472172176775487, 0.0233436625590571, true, 0, null, 0);
            Debug.Print("Solid body surface trim feature? " + swFeatureManager.SolidForTrim);
            swFeatureManager.SolidForTrim = true;
            Debug.Print("Solid body surface trim feature? " + swFeatureManager.SolidForTrim);
            swFeature = (Feature)swFeatureManager.PostTrimSurface(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Solid Body Surface Trim Feature Example (VBA)

This example shows how to create a solid body surface trim feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Creates Surface-Trim1.
' 3. Expand and examine Solid Bodies(1) in the FeatureManager design tree
'    and examine the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim status As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SurfaceTrimFeature.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager

    ' Select surface features
    status = swModelDocExt.SelectByID("", "SURFACEBODY", -4.46486526100784E-02, 2.18350174377093E-02, 1.23754341749418E-02, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "SURFACEBODY", -8.15686270678384E-03, 4.15839719953865E-02, 2.42402652081068E-02, True, 0, Nothing, 0)

    ' Select trimming surfaces to create solid body surface trim feature
    status = swFeatureManager.PreTrimSurface(True, True, False, True)
    status = swModelDocExt.SelectByID("", "SURFACEBODY", 5.9504253577245E-03, 4.13800871671199E-02, 2.48740287174201E-02, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "SURFACEBODY", -0.037205042299604, 3.43527327176432E-02, 1.23446167727934E-02, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "SURFACEBODY", -1.04497983190015E-02, -4.72172176775487E-02, 2.33436625590571E-02, True, 0, Nothing, 0)
    Debug.Print "Solid body surface trim feature? " & swFeatureManager.SolidForTrim
    swFeatureManager.SolidForTrim = True
    Debug.Print "Solid body surface trim feature? " & swFeatureManager.SolidForTrim
    Set swFeature = swFeatureManager.PostTrimSurface(True)

End Sub
```

Create Solid Body Surface Trim Feature Example (VB.NET)

This example shows how to create a solid body surface trim feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part to open exists.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Opens the specified part.
' 2. Creates Surface-Trim1.
' 3. Expand and examine Solid Bodies(1) in the FeatureManager
'    design tree and examine the Immediate window.
'-----
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim status As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SurfaceTrimFeature.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager

        ' Select surface features
        status = swModelDocExt.SelectByID("", "SURFACEBODY", -0.0446486526100784, 0.0218350174377093, 0.0123754341749418, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "SURFACEBODY", -0.00815686270678384, 0.0415839719953865, 0.0242402652081068, True, 0, Nothing, 0)

        ' Select trimming surfaces to create solid body surface trim feature
        status = swFeatureManager.PreTrimSurface(True, True, False, True)
        status = swModelDocExt.SelectByID("", "SURFACEBODY", 0.0059504253577245, 0.0413800871671199, 0.0248740287174201, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "SURFACEBODY", -0.037205042299604, 0.0343527327176432, 0.0123446167727934, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "SURFACEBODY", -0.0104497983190015, -0.047217217677548703, 0.0233436625590571, True, 0, Nothing, 0)
        Debug.Print("Solid body surface trim feature? " & swFeatureManager.SolidForTrim)
        swFeatureManager.SolidForTrim = True
        Debug.Print("Solid body surface trim feature? " & swFeatureManager.SolidForTrim)
        swFeature = swFeatureManager.PostTrimSurface(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Space Parameter Curve on Surface Example (VBA)

This example shows how to create a space parameter (SP) curve on a surface. This example also shows how to pack integers into doubles.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Select a face.
'
' Postconditions:
' 1. Creates a 3D sketch that contains a curve lying
'    on selected face.
' 2. Examine the graphics area.
'
' NOTES:
' * Sketch contains a tessellated approximation of
'   the curve lying on the selected face
' * Control points are a random selection of U-V-W
'   values.
'-----
```

Option Explicit

```
' Define two types
Type DoubleRec
    dValue As Double
End Type
```

```
Type Int2Rec
    iLower As Long ' C int has 4 bytes
    iUpper As Long
End Type
```

```
Sub CreateTessCurve(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTrimCurve As SldWorks.Curve)
```

```
    Const nChordTol          As Double = 0.001
    Const nLengthTol         As Double = 0.001
    Dim nStartParam           As Double
    Dim nEndParam             As Double
    Dim bIsClosed              As Boolean
    Dim bIsPeriodic            As Boolean
    Dim vStartPt              As Variant
    Dim vEndPt                As Variant
    Dim vTessPts               As Variant
    Dim swSketchSeg            As SldWorks.SketchSegment
    Dim bRet                   As Boolean
    Dim i                      As Long
```

```
    ' Not really needed because curve is a trimmed curve,
    ' so could pass in trim points as parameters
    bRet = swTrimCurve.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic): Debug.Assert bRet
```

```
    vStartPt = swTrimCurve.Evaluate(nStartParam)
    vEndPt = swTrimCurve.Evaluate(nEndParam)
```

```
    vTessPts = swTrimCurve.GetTessPts(nChordTol, nLengthTol, (vStartPt), (vEndPt))
```

```
    swModel.Insert3DSketch2 False
```

```
    swModel.SetAddToDB True
```

```
    swModel.SetDisplayWhenAdded False
```

```
    ' Disable Visual Basic range checking because tessellation points
    ' may not be a multiple of 6
```

```
On Error Resume Next
```

```
For i = 0 To UBound(vTessPts) Step 3
```

```
    Set swSketchSeg = swModel.CreateLine2(vTessPts(i + 0), vTessPts(i + 1), vTessPts(i + 2), vTessPts(i + 3), vTessPts(i + 4), vTessPts(i + 5))
Next i
```

```
On Error GoTo 0
```

```
swModel.SetDisplayWhenAdded True
```

```
swModel.SetAddToDB False
```

```
swModel.Insert3DSketch2 True
```

```
bRet = swModel.EditRebuild3: Debug.Assert bRet
```

```
End Sub
```

```
Sub main()
```

```
    Dim swApp                As SldWorks.SldWorks
    Dim swModeler             As SldWorks.Modeler
    Dim swModel               As SldWorks.ModelDoc2
    Dim swSelMgr               As SldWorks.SelectionMgr
    Dim swFace                As SldWorks.Face2
    Dim swSurr                As SldWorks.Surface
    Dim vProps                 As Variant
    Dim nProp(1)               As Double
    Dim i2rDim_Order           As Int2Rec
    Dim i2rNumCtrlPt_Period    As Int2Rec
    Dim nDummy1                As DoubleRec
    Dim nDummy2                As DoubleRec
    Dim swCurve                As SldWorks.Curve
```

```

Dim nKnot(9)                As Double
Dim vKnot                   As Variant
Dim nCtrlPt(8)              As Double
Dim vCtrlPt                 As Variant

Set swApp = Application.SldWorks
Set swModeler = swApp.GetModeler
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFace = swSelMgr.GetSelectedObject6(1, -1)
Set swSurf = swFace.GetSurface

' Dimension of control points 'dim':
' * For rational curves 'dim' = 3
' * For non-rational curves 'dim' = 2
i2rDim_Order.iLower = 3

' Order of the curve 'order':
' * The order of the curve = degree + 1
' * The minimum order is 2
i2rDim_Order.iUpper = 3

' Number of control points 'nctrl': 'nctrl' >= 'order'
i2rNumCtrlPt_Period.iLower = 3

' Periodic flag 'period':
' * For periodic curve = 1
' * For non-periodic curve = 0
i2rNumCtrlPt_Period.iUpper = 0
LSet nDummy1 = i2rDim_Order
LSet nDummy2 = i2rNumCtrlPt_Period
nProp(0) = nDummy1.dValue
nProp(1) = nDummy2.dValue

vProps = nProp

' Knot vector 'knots': The knot values must form a non-decreasing sequence
nKnot(0) = 0.09
nKnot(1) = 0.1
nKnot(2) = 0.2
nKnot(3) = 0.3
nKnot(4) = 0.4
nKnot(5) = 0.5
nKnot(6) = 0.6
nKnot(7) = 0.7
nKnot(8) = 0.8
nKnot(9) = 0.9
vKnot = nKnot

' Control points 'ctrl':
' * For non-rational curves, the control
'   points are points in the parameter space of 'surf';
'   they must be supplied as [u0,v0,u1,v1...]
' * For rational curves each vector contains a point in parameter space
'   followed by a weight for the point; the points are supplied
' [u0,v0,w0,u1,v1,w1...]; the weights must be positive
nCtrlPt(0) = 168.441616909048 / 1000#: nCtrlPt(1) = 150.079444300104 / 1000#: nCtrlPt(2) = 361.067614130377 / 1000#
nCtrlPt(3) = 200.587021030302 / 1000#: nCtrlPt(4) = 133.788963935729 / 1000#: nCtrlPt(5) = 15.7123672133821 / 1000#
nCtrlPt(6) = 454.553312070037 / 1000#: nCtrlPt(7) = 78.0558561766611 / 1000#: nCtrlPt(8) = 1.00713763464455 / 1000#
vCtrlPt = nCtrlPt

' Eventually goes down to CRSPCU, which is equivalent to:
' * PK_CURVE_embed_in_surf
' * PK_SPCURVE_create
Set swCurve = swModeler.CreatePCurve(swSurf, (vProps), (vKnot), (vCtrlPt))
Debug.Assert Not swCurve Is Nothing
Debug.Assert swCurve.IsTrimmedCurve

CreateTessCurve swApp, swModel, swCurve

End Sub

```

Create Specific Dimension in a Sketch Example (C#)

This example shows how to add an angular display dimension to a sketch.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\box.sldprt.
//
// Postconditions:
// 1. Edits Sketch1.
// 2. Selects two intersecting lines.
// 3. Creates an angular display dimension at the specified location in the
//    sketch.
// 4. Inserts Sketch1.
// 5. Inspect Sketch1 in the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes to it.
// -----

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;
using System.Data;
using System.Diagnostics;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateAngDim_CSharp
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DisplayDimension myDisplayDim;
        bool boolstatus;
        int err;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);

            Part.EditSketch();

            boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", -0.0509671483361161, -0.0109842011554073, 0.0296211826,
            boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", -0.0770411440149667, 0.00496030150977761, 0.0325476156,
            myDisplayDim = (DisplayDimension)Part.Extension.AddSpecificDimension(-0.0456250220540824, 0, 0.00150965590938767, (int)5,
            Part.SketchManager.InsertSketch(true);

        }

        // The SldWorks swApp variable is pre-assigned for you.
        public SldWorks swApp;
    }
}
```

Create Specific Dimension in a Sketch Example (VBA)

This example shows how to add an angular display dimension to a sketch.

Preconditions: Open *public_documents\samples\tutorial\api\box.sldprt*.

Postconditions:

1. Edits **Sketch1**.
2. Selects two intersecting lines.
3. Creates an angular display dimension at the specified location in the sketch.
4. Inserts **Sketch1**.
5. Inspect **Sketch1** in the graphics area.

NOTE: Because the model is used elsewhere, do not save changes to it.

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim error As Long

Option Explicit
Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

    Part.EditSketch

    boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", -5.09671483361161E-02, -1.09842011554073E-02,
2.96211826739789E-02, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", -7.70411440149667E-02, 4.96030150977761E-03,
3.25476150359756E-02, True, 0, Nothing, 0)

    Set myDisplayDim = Part.Extension.AddSpecificDimension(-4.56250220540824E-02, 0, 1.50965590938767E-03, swAngularDimension,
error)

    Part.SketchManager.InsertSketch True
End Sub
```

Create Specific Dimension in a Sketch Example (VB.NET)

This example shows how to add an angular display dimension to a sketch.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\box.sldprt.
'
' Postconditions:
' 1. Edits Sketch1.
' 2. Selects two intersecting lines.
' 3. Creates an angular display dimension at the specified location in the
'    sketch.
' 4. Inserts Sketch1.
' 5. Inspect Sketch1 in the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim myDisplayDim As DisplayDimension
    Dim boolstatus As Boolean
    Dim err As Integer
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Part.EditSketch
```

```
        boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", -0.0509671483361161, -0.0109842011554073, 0.02962118267397
        boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", -0.0770411440149667, 0.00496030150977761, 0.03254761503597
        myDisplayDim = Part.Extension.AddSpecificDimension(-0.0456250220540824, 0, 0.00150965590938767, swDimensionType_e.swAngularI
```

```
        Part.SketchManager.InsertSketch(True)
```

```
    End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Create a SpeedPak Configuration Example (C#)

This example shows how to create a SpeedPak configuration for the active configuration.

```
//-----  
// Preconditions: Open an assembly.  
//  
// Postconditions:  
// 1. Adds active_configuration (Default_speedpak) is added  
// ConfigurationManager.  
// 2. Click the ConfigurationManager tab and expand Default.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace SpeedpakConfigurationCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            bool boolstatus = false;  
            ConfigurationManager swConfigMgr = default(ConfigurationManager);  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;  
  
            // Add a SpeedPak configuration with a part/body  
            // selection threshold of 0.5 to the active configuration  
            swConfigMgr.AddSpeedPak2(1, 0.5);  
            boolstatus = swModel.ForceRebuild3(false);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
  
        public SldWorks swApp;  
    }  
}
```

Create a SpeedPak Configuration Example (VBA)

This example shows how to create a SpeedPak configuration for the active configuration.

```
'-----  
' Preconditions: Open an assembly.  
'  
' Postconditions:  
' 1. Adds active_configuration (Default_speedpak) to the  
'   ConfigurationManager.  
' 2. Click the ConfigurationManager tab and expand Default.  
'-----
```

Option Explicit

Sub main()

```
    Dim SwApp As SldWorks.SldWorks  
    Dim Part As SldWorks.ModelDoc2  
    Dim boolstatus As Boolean  
    Dim ConfigMgr As SldWorks.ConfigurationManager  
  
    Set SwApp = Application.SldWorks  
    If SwApp Is Nothing Then Exit Sub  
  
    Set Part = SwApp.ActiveDoc  
    Set ConfigMgr = Part.ConfigurationManager
```

```
    ' Add a SpeedPak configuration with a part/body selection threshold of 0.5 to the active configuration  
    ConfigMgr.AddSpeedPak2 1, 0.5  
    boolstatus = Part.ForceRebuild3(False)
```

End Sub

Create a SpeedPak Configuration Example (VB.NET)

This example shows how to create a SpeedPak configuration for the active configuration.

```
'-----  
' Preconditions: Open an assembly.  
'  
' Postconditions:  
' 1. Adds active_configuration (Default_speedpak) to the  
'   ConfigurationManager.  
' 2. Click the ConfigurationManager tab and expand Default.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim boolstatus As Boolean  
        Dim swConfigMgr As ConfigurationManager  
  
        swModel = swApp.ActiveDoc  
        swConfigMgr = swModel.ConfigurationManager  
  
        ' Add a SpeedPak configuration with a part/body  
        ' selection threshold of 0.5 to the active configuration  
        swConfigMgr.AddSpeedPak2(1, 0.5)  
        boolstatus = swModel.ForceRebuild3(False)  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks
```

```
End Class
```

Create SpeedPak for Subassemblies Example (C#)

This example shows how to:

- create SpeedPak configurations for subassemblies.
- switch between the SpeedPak configuration and the parent configuration of the SpeedPak configuration.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly.
// 2. Deletes the Coordinate System1 feature to avoid errors.
// 3. Selects the arm1 and arm2 subassemblies.
// 4. Checks to see if either component has a SpeedPak configuration.
// 5. Creates graphics-only SpeedPak for each selected component.
// 6. At System.Diagnostics.Debugger.Break, examine the FeatureManager
// design tree to verify that the icons for arm1 and arm2 indicate
// SpeedPak, then press F5.
// 7. Switches SpeedPak to each selected component's parent configuration.
// 8. At System.Diagnostics.Debugger.Break, examine the FeatureManager
// design tree to verify that SpeedPak switched to the parent configuration
// of arm1 and arm2, then press F5.
// 9. Switches each component back to SpeedPak.
// 10. Examine the Immediate window and FeatureManager design tree to verify
// that arm1 and arm2 are SpeedPak.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using System;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            AssemblyDoc swAssembly;
            SelectionMgr swSelMgr;
            Component2 swComponent;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName;
            object[] selComponents = new object[2];
            int i = 0;
            int count = 0;
            bool speedPakExists = false;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\wrench.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
rrors, ref warnings);
            swAssembly = (AssemblyDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Select and delete Coordinate System1 to avoid errors
            status = swModelDocExt.SelectByID2("Coordinate System1", "COORDSYS", 0, 0, 0, false, 0, null, 0);
            swModel.EditDelete();

            //Select the two subassemblies for which to create SpeedPak configurations
            status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            selComponents[0] = swSelMgr.GetSelectedObject6(1, -1);
            status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            selComponents[1] = swSelMgr.GetSelectedObject6(2, -1);
            count = swSelMgr.GetSelectedObjectCount2(-1);

            //Get whether any of the selected components already
            //have SpeedPak configurations
            for (i = 0; i <= count - 1; i++)
            {
                swComponent = (Component2)selComponents[i];
                speedPakExists = swComponent.IsSpeedPak;
                if ((speedPakExists))
                {
                    Debug.Print("SpeedPak already exists for component(" + i + ")");
                }
            }

            //Create graphics-only SpeedPak for the selected components
            status = swAssembly.CreateSpeedPak(2);
            Debug.Print("SpeedPak created for selected components? " + status);

            System.Diagnostics.Debugger.Break();
            //Examine the FeatureManager design tree to verify that the
            //icons for arm1 and arm2 indicate SpeedPak, then press F5

            //Switch SpeedPak to the parent configurations of each selected components
            status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swAssembly.SetSpeedPakToParent();
            Debug.Print("SpeedPak switched to the parent configuration of each selected component? " + status);

            System.Diagnostics.Debugger.Break();
            //Examine the FeatureManager design tree to verify
            //that SpeedPak switched to the parent configurations of arm1 and
            //arm2

            //Switch the selected components to SpeedPak
            status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swAssembly.UseSpeedPak();
        }
    }
}
```

```
        Debug.Print("Switched the selected components to SpeedPak? " + status);
    }
    public SldWorks swApp;
}
```

Create SpeedPak for Subassemblies Example (VBA)

This example shows how to:

- create SpeedPak configurations for subassemblies.
- switch between the SpeedPak configuration and the parent configuration of the SpeedPak configuration.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Deletes the Coordinate System1 feature to avoid errors.
' 3. Selects the arm1 and arm2 subassemblies.
' 4. Checks to see if either component has a SpeedPak configuration.
' 5. Creates graphics-only SpeedPak for each selected component.
' 6. At Stop, examine the FeatureManager design tree to verify that the
'    icons for arm1 and arm2 indicate SpeedPak, then press F5.
' 7. Switches SpeedPak to each selected component's parent configuration.
' 8. At Stop, examine the FeatureManager design tree to verify that
'    SpeedPak switched to the parent configuration of arm1 and
'    arm2, then press F5.
' 9. Switches each component back to SpeedPak.
' 10. Examine the Immediate window and FeatureManager design tree to verify
'     that arm1 and arm2 are SpeedPak.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim swSelMgr As SldWorks.SelectionMgr
Dim swComponent As SldWorks.Component2
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim selComponents(1) As Object
Dim i As Long
Dim count As Long
Dim speedPakExists As Boolean

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssembly = swModel
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    'Select and delete Coordinate System1 to avoid errors
    status = swModelDocExt.SelectByID2("Coordinate System1", "COORDSYS", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditDelete

    'Select the two subassemblies for which to create SpeedPak configurations
    status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set selComponents(0) = swSelMgr.GetSelectedObject6(1, -1)
    status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    Set selComponents(1) = swSelMgr.GetSelectedObject6(2, -1)
    count = swSelMgr.GetSelectedObjectCount2(-1)

    'Get whether any of the selected components already
    'have SpeedPak configurations
    For i = 0 To count - 1
        Set swComponent = selComponents(i)
        speedPakExists = swComponent.IsSpeedPak
        If (speedPakExists) Then
            Debug.Print ("SpeedPak already exists for component(" & i & ")")
        End If
    Next

    'Create graphics-only SpeedPak for the selected components
    status = swAssembly.CreateSpeedPak(2)
    Debug.Print ("SpeedPak created for selected components? " & status)

    Stop

    'Examine the FeatureManager design tree to verify that the
    'icons for arm1 and arm2 indicate SpeedPak, then press F5

    'Switch SpeedPak to the parent configurations of each selected components
    status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swAssembly.SetSpeedPakToParent
    Debug.Print ("SpeedPak switched to the parent configuration of each selected component? " & status)

    Stop

    'Examine the FeatureManager design tree to verify
    'that SpeedPak switched to the parent configurations of arm1 and
    'arm2

```

```
'Switch the selected components to SpeedPak
status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
status = swAssembly.UseSpeedPak
Debug.Print ("Switched the selected components to SpeedPak? " & status)
```

End Sub

Create SpeedPak for Subassemblies Example (VB.NET)

This example shows how to:

- create SpeedPak configurations for subassemblies.
- switch between the SpeedPak configuration and the parent configuration of the SpeedPak configuration.

Preconditions:

1. Verify that the specified assembly to open exists.
2. Open the Immediate window.

Postconditions:

1. Opens the specified assembly.
2. Deletes the **Coordinate System1** feature to avoid errors.
3. Selects the **arm1** and **arm2** subassemblies.
4. Checks to see if either component has a SpeedPak configuration.
5. Creates graphics-only SpeedPak for each selected component.
6. At Stop, examine the FeatureManager design tree to verify that the icons for **arm1** and **arm2** indicate SpeedPak, then press F5.
7. Switches SpeedPak to each selected component's parent configuration.
8. At Stop, examine the FeatureManager design tree to verify that SpeedPak switched to the parent configuration of **arm1** and **arm2**, then press F5.
9. Switches each component back to SpeedPak.
10. Examine the Immediate window and FeatureManager design tree to verify that **arm1** and **arm2** are SpeedPak.

NOTE: Because the assembly is used elsewhere, do not save changes.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
Dim swModel As ModelDoc2
Dim swModelDocExt As ModelDocExtension
Dim swAssembly As AssemblyDoc
Dim swSelMgr As SelectionMgr
Dim swComponent As Component2
Dim status As Boolean
Dim errors As Integer
Dim warnings As Integer
Dim fileName As String
Dim selComponents(1) As Object
Dim i As Integer
Dim count As Integer
Dim speedPakExists As Boolean

Sub main()

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\wrench.sldasm"
    swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    swAssembly = swModel
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager

    'Select and delete Coordinate System1 to avoid errors
    status = swModelDocExt.SelectByID2("Coordinate System1", "COORDSYS", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditDelete()

    'Select the two subassemblies for which to create SpeedPak configurations
    status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    selComponents(0) = swSelMgr.GetSelectedObject6(1, -1)
    status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    selComponents(1) = swSelMgr.GetSelectedObject6(2, -1)
    count = swSelMgr.GetSelectedObjectCount2(-1)

    'Get whether any of the selected components already
    'have SpeedPak configurations
    For i = 0 To count - 1
        swComponent = selComponents(i)
        speedPakExists = swComponent.IsSpeedPak
        If (speedPakExists) Then
            Debug.Print("SpeedPak already exists for component(" & i & ")")
        End If
    Next

    'Create graphics-only SpeedPak for the selected components
    status = swAssembly.CreateSpeedPak(2)
    Debug.Print("SpeedPak created for selected components? " & status)

    Stop

    'Examine the FeatureManager design tree to verify that the
    'icons for arm1 and arm2 indicate SpeedPak, then press F5

    'Switch SpeedPak to the parent configurations of each selected components
    status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swAssembly.SetSpeedPakToParent
    Debug.Print("SpeedPak switched to the parent configuration of each selected component? " & status)

    Stop

    'Examine the FeatureManager design tree to verify
    'that SpeedPak switched to the parent configurations of arm1 and
    'arm2

    'Switch the selected components to SpeedPak
    status = swModelDocExt.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swAssembly.UseSpeedPak
    Debug.Print("Switched the selected components to SpeedPak? " & status)

End Sub
```

''' <summary>

```
''' The Sldworks swApp variable is pre-assigned for you.
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Spherical Surface, Trimmed Sheet, and Feature from Body Example (VBA)

This example shows how to create a spherical surface, a trimmed sheet, and a feature from a body.

```
'=====
```

```
'Preconditions:
```

```
'1. Open a new part.
```

```
'2. Run the macro to the Stop.
```

```
'3. Press F5 to create the imported feature.
```

```
'
```

```
'Postconditions:
```

```
'1. Creates a temporary body and displays it.
```

```
'2. Uses the temporary body to create Surface-Imported1
```

```
'    in the FeatureManager design tree.
```

```
'=====
```

```
Const RADIUS As Double = 0.01
```

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swPart As SldWorks.PartDoc
```

```
Dim swSurf As SldWorks.Surface
```

```
Dim swBody As SldWorks.Body2
```

```
Dim swFeat As SldWorks.Feature
```

```
Dim swModeler As SldWorks.Modeler
```

```
Option Explicit
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
```

```
    Set swPart = swApp.ActiveDoc
```

```
    Set swModeler = swApp.GetModeler
```

```
    Dim dCenter(2) As Double
```

```
    dCenter(0) = 0: dCenter(1) = 0: dCenter(2) = 0
```

```
    Dim dAxis(2) As Double
```

```
    dAxis(0) = 0: dAxis(1) = 0: dAxis(2) = 1
```

```
    Dim dRef(2) As Double
```

```
    dRef(0) = 1: dRef(1) = 0: dRef(2) = 0
```

```
    Set swSurf = swModeler.CreateSphericalSurface2(dCenter, dAxis, dRef, RADIUS)
```

```
    ' No trimming curves are required, so pass one Nothing in the Curves parameter array
```

```
    ' of ISurface::CreateTrimmedSheet5
```

```
    Dim ZeroTrimmingCurves(0 To 0) As SldWorks.Curve
```

```
    Set ZeroTrimmingCurves(0) = Nothing
```

```
    Set swBody = swSurf.CreateTrimmedSheet5(ZeroTrimmingCurves, True, 0.00001)
```

```
    swBody.Display3 swPart, RGB(255, 255, 0), swTempBodySelectOptions_e.swTempBodySelectable
```

```
    Stop
```

```
    Set swFeat = swPart.CreateFeatureFromBody3(swBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck +  
    swCreateFeatureBodyOpts_e.swCreateFeatureBodySimplify)
```

```
End Sub
```

Create Spiral Example (C#)

This example shows how to create a spiral.

```
//-----  
// Preconditions: Specified part template exists.  
//  
// Postconditions:  
// 1. Opens a new part.  
// 2. Selects Front Plane on which to create a circle.  
// 3. Creates a circle.  
// 4. Creates a spiral using the circle.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace SpiralCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            SketchManager swSketchMgr = default(SketchManager);  
            SketchSegment swSketchSegment = default(SketchSegment);  
            bool status = false;  
  
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prtdot", 0, 0, 0);  
            //Select Front Plane, create circle, and create  
            //spiral using circle  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0517981810568133, 0.0505753331558577, 0.0012310671470727, false, 0, null, 0);  
            swModel.ClearSelection2(true);  
            swSketchMgr = (SketchManager)swModel.SketchManager;  
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.021866, 0.001156, 0.0);  
            swModel.InsertHelix(false, true, false, false, (int)swHelixDefinedBy_e.swHelixDefinedBySpiral, 0, 0.04, 2, 0, 4.712388980385);  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Create Spiral Example (VBA)

This example shows how to create a spiral.

```
'-----  
' Preconditions: Specified part template exists.  
,  
' Postconditions:  
' 1. Opens a new part.  
' 2. Selects Front Plane on which to create a circle.  
' 3. Creates a circle.  
' 4. Creates a spiral using the circle.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchMgr As SldWorks.SketchManager  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim status As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)  
  
    'Select Front Plane, create circle, and create  
    'spiral using circle  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", -5.17981810568133E-02, 5.05753331558577E-02, 1.2310671470727E-03, False, 0, Nothing, 0)  
    swModel.ClearSelection2 True  
    Set swSketchMgr = swModel.SketchManager  
    Set swSketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.021866, 0.001156, 0#)  
    swModel.InsertHelix False, True, False, False, swHelixDefinedBy_e.swHelixDefinedBySpiral, 0, 0.04, 2, 0, 4.712388980385  
  
End Sub
```

Create Spiral Example (VB.NET)

This example shows how to create a spiral.

```
.....
' Preconditions: Specified part template exists.
'
' Postconditions:
' 1. Opens a new part.
' 2. Selects Front Plane on which to create a circle.
' 3. Creates a circle.
' 4. Creates a spiral using the circle.
.....
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
Public Sub Main()

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSketchMgr As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim status As Boolean

    swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prt", 0, 0, 0)
    'Select Front Plane, create circle, and create
    'spiral using circle
    swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0517981810568133, 0.0505753331558577, 0.0012310671470727, False, 0, Nothing, 0)
    swModel.ClearSelection2(True)
    swSketchMgr = swModel.SketchManager
    swSketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.021866, 0.001156, 0.0#)
    swModel.InsertHelix(False, True, False, False, swHelixDefinedBy_e.swHelixDefinedBySpiral, 0, 0.04, 2, 0, 4.712388980385)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Create Split Feature Example (C#)

This example shows how to create a Split feature.

```
//-----
// Preconditions:
// 1. Open a part document that contains a body that is bisected by the
//    Top Plane.
// 2. Verify that c:\temp exists.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Creates Split1.
// 2. Saves a split body to c:\temp\Body1.sldprt.
// 3. Inspect the Immediate window, FeatureManager design tree, graphics area,
//    and c:\temp.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateSplitBody2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        ModelDocExtension swModelDocExt;
        Feature swFeat;
        FeatureManager swFeatMgr;
        SplitBodyFeatureData swSplitBodyFeat;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeatMgr = swModel.FeatureManager;

            //Select the cutting tool
            boolstatus = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, true, 0, null, 0);

            object vBodyNames = null;
            object[] bodiesToMark = new Body[2];
            string[] bodyNames = new string[2];
            object[] bodyOrigins = new Vertex[2];

            //Get bodies that will result from the split operation
            object[] vResultingBodies = null;
            vResultingBodies = (object[])swFeatMgr.PreSplitBody2();

            swModel.ClearSelection2(true);

            //Set up the arrays for the post-split operation

            //If you do not want to assign body origin, set it to nothing so that the default origin is used
            bodyOrigins[0] = null;
            bodyOrigins[1] = null;

            bodiesToMark[0] = vResultingBodies[0];
            bodiesToMark[1] = vResultingBodies[1];

            //Save the first body to a separate part document
            //Substitute the name of the actual folder where to save the body
            bodyNames[0] = "c:\\temp\\Body1.sldprt";
            //Do not save the second body
            bodyNames[1] = "";

            DispatchWrapper[] preSplitBodies = null;
            preSplitBodies = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray((bodiesToMark));
            vBodyNames = bodyNames;
            DispatchWrapper[] originsToUse = null;
            originsToUse = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray((bodyOrigins));

            //Create the Split feature, consuming all split bodies
            swFeat = swFeatMgr.PostSplitBody2((preSplitBodies), true, (originsToUse), (vBodyNames), "");

            if ((swFeat != null))
            {
                Debug.Print("Split feature: " + swFeat.Name);
                swSplitBodyFeat = (SplitBodyFeatureData)swFeat.GetDefinition();
                Debug.Print("Bodies consumed? " + swSplitBodyFeat.Consume);
                Debug.Print("");
            }
        }

        public DispatchWrapper[] ObjectArrayToDispatchWrapperArray(object[] Objects)
        {
            int ArraySize = 0;
            ArraySize = Objects.GetUpperBound(0);
            DispatchWrapper[] d = new DispatchWrapper[ArraySize + 1];
            int ArrayIndex = 0;
            for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
            {
                d[ArrayIndex] = new DispatchWrapper(Objects[ArrayIndex]);
            }
            return d;
        }

        public SldWorks swApp;
```


Create Split Feature Example (VBA)

This example shows how to create a Split feature.

```

'-----
' Preconditions:
' 1. Open a part document that contains a body that is bisected by
'    Top Plane.
' 2. Verify that c:\temp exists.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Creates Split1.
' 2. Saves a split body to c:\temp\Body1.sldprt.
' 3. Inspect the Immediate window, FeatureManager design tree, graphics area,
'    and c:\temp.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeat As SldWorks.Feature
Dim swFeatMgr As SldWorks.FeatureManager
Dim swSplitBodyFeat As SldWorks.SplitBodyFeatureData
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Set swFeatMgr = swModel.FeatureManager

    'Select the cutting tool
    boolstatus = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

    'Get bodies resulting from the split operation
    Dim vResultingBodies As Variant
    vResultingBodies = swFeatMgr.PreSplitBody2

    swModel.ClearSelection2 True

    Dim vBodiesToMark As Variant
    Dim vBodyNames As Variant
    Dim vBodyOrigins As Variant
    Dim bodiesToMark(1) As Object
    Dim bodyNames(1) As String
    Dim bodyOrigins(1) As Object

    'Set up the arrays for the post-split operation
    'Specify the origins of the bodies to save; specify Nothing to use the default origins
    Set bodyOrigins(0) = Nothing
    Set bodyOrigins(1) = Nothing

    Set bodiesToMark(0) = vResultingBodies(0)
    Set bodiesToMark(1) = vResultingBodies(1)

    'Save the first body to a separate part document
    'Substitute the name of the actual folder where to save the body
    bodyNames(0) = "c:\temp\Body1.sldprt"

    'Do not save the second body
    bodyNames(1) = ""

    vBodiesToMark = bodiesToMark
    vBodyNames = bodyNames
    vBodyOrigins = bodyOrigins

    'Create the Split feature, consuming all split bodies
    Set swFeat = swFeatMgr.PostSplitBody2((vBodiesToMark), True, (vBodyOrigins), (vBodyNames), "")

    If (Not swFeat Is Nothing) Then
        Debug.Print "Split feature: " & swFeat.Name
        Set swSplitBodyFeat = swFeat.GetDefinition
        Debug.Print "Consumed? " & swSplitBodyFeat.Consume
        Debug.Print " "
    End If
End Sub

```

Create Split Feature Example (VB.NET)

This example shows how to create a Split feature.

```

'-----
' Preconditions:
' 1. Open a part document that contains a body that is bisected by
'    Top Plane.
' 2. Verify that c:\temp exists.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Creates Split1.
' 2. Saves a split body to c:\temp\Body1.sldprt.
' 3. Inspect the Immediate window, FeatureManager design tree, graphics area,
'    and c:\temp.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swModelDocExt As ModelDocExtension
    Dim swFeat As Feature
    Dim swFeatMgr As FeatureManager
    Dim swSplitBodyFeat As SplitBodyFeatureData
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swFeatMgr = swModel.FeatureManager

        'Select the cutting tool
        boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

        'Get bodies resulting from the split operation
        Dim vResultingBodies(3) As Object
        vResultingBodies = swFeatMgr.PreSplitBody2

        swModel.ClearSelection2(True)

        Dim vBodyNames As Object
        Dim bodiesToMark(1) As Body2
        Dim bodyNames(1) As String
        Dim bodyOrigins(1) As Vertex

        'Set up the arrays for the post-split operation

        'Assign the origins of bodies to save; set to nothing to use default origins
        bodyOrigins(0) = Nothing
        bodyOrigins(1) = Nothing

        bodiesToMark(0) = vResultingBodies(0)
        bodiesToMark(1) = vResultingBodies(1)

        'Save the first body to a separate part document
        bodyNames(0) = "c:\temp\Body1.sldprt"
        'Do not save the second body
        bodyNames(1) = ""

        Dim preSplitBodies() As DispatcherWrapper
        preSplitBodies = ObjectArrayToDispatcherWrapperArray((bodiesToMark))
        vBodyNames = bodyNames
        Dim originsToUse() As DispatcherWrapper
        originsToUse = ObjectArrayToDispatcherWrapperArray((bodyOrigins))

        'Create the Split feature, consuming all split bodies
        swFeat = swFeatMgr.PostSplitBody2((preSplitBodies), True, (originsToUse), (vBodyNames), "")

        If (Not swFeat Is Nothing) Then
            Debug.Print("Split feature: " & swFeat.Name)
            swSplitBodyFeat = swFeat.GetDefinition
            Debug.Print("Bodies consumed? " & swSplitBodyFeat.Consume)
            Debug.Print(" ")
        End If

    End Sub

    Function ObjectArrayToDispatcherWrapperArray(ByVal Objects As Object()) As DispatcherWrapper()
        Dim ArraySize As Integer
        ArraySize = Objects.GetUpperBound(0)
        Dim d(ArraySize) As DispatcherWrapper
        Dim ArrayIndex As Integer
        For ArrayIndex = 0 To ArraySize
            d(ArrayIndex) = New DispatcherWrapper(Objects(ArrayIndex))
        Next
        Return d
    End Function

    Public swApp As SldWorks
End Class

```

Create Standard Mates Example (C#)

This example shows how to create a variety of standard mates.

```
//-----
// Preconditions: Ensure the specified file paths exist.
//
// Postconditions: Inspect the Mates folder in the FeatureManager design tree.
//
// NOTE: Because the models are used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace StandardMates_CSharp
{
    partial class SolidWorksMacro
    {
        private AssemblyDoc swAssembly;
        private string AssemblyTitle;
        private ModelDoc2 tmpObj;
        private Component2 swInsertedComponent;
        private int errors;
        private ModelDoc2 Part;
        private double[] TransformData = new double[16];
        private object TransformDataVariant;
        private dynamic swMathUtil;
        private MathTransform swTransform;
        private TangentMateFeatureData MateData;
        private object[] EntitiesToMate = new object[2];
        private object EntitiesToMateVar;
        private Feature MateFeature;
        private PerpendicularMateFeatureData PerpMateData;
        private CoincidentMateFeatureData CoincMateData;
        private ParallelMateFeatureData ParMateData;
        private ConcentricMateFeatureData ConcMateData;
        private LockMateFeatureData LMateData;
        private bool boolstatus;
        private int longstatus;
        private int longwarnings;
        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\assem20.sldasm");

            swAssembly = (AssemblyDoc)Part;
            swApp.ActivateDoc2("assem20.sldasm", false, longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            AssemblyTitle = Part.GetTitle();

            tmpObj = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\shaft.sldprt");
            Part = (ModelDoc2)swApp.ActiveDoc3(AssemblyTitle, true, 0, ref errors);

            swInsertedComponent = swAssembly.AddComponent5("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\shaft.sldprt");
            swApp.CloseDoc("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\shaft.sldprt");

            TransformData[0] = 1;
            TransformData[1] = 0;
            TransformData[2] = 0;
            TransformData[3] = 0;
            TransformData[4] = 1;
            TransformData[5] = 0;
            TransformData[6] = 0;
            TransformData[7] = 0;
            TransformData[8] = 1;
            TransformData[9] = 0.29642267129384;
            TransformData[10] = 0.0420506109250709;
            TransformData[11] = -0.187506963149644;
            TransformData[12] = 1;
            TransformData[13] = 0;
            TransformData[14] = 0;
            TransformData[15] = 0;

            TransformDataVariant = TransformData;

            swMathUtil = swApp.GetMathUtility();

            swTransform = swMathUtil.CreateTransform((TransformDataVariant));
            boolstatus = swInsertedComponent.SetTransformAndSolve2(swTransform);

            boolstatus = Part.Extension.SelectByRay(0.0443527596829085, 0.0297941775455115, 0.00219712535516692, -0.400036026779312, -0.400036026779312,
            boolstatus = Part.Extension.SelectByRay(0.130082347379926, 0.0512737883206569, -0.0238688162796734, -0.400036026779312,

            // Create TangentMateFeatureData

```

```

MateData = (TangentMateFeatureData)swAssembly.CreateMateData(4);

// Set the Entities To Mate
EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);

EntitiesToMateVar = EntitiesToMate;
MateData.EntitiesToMate = (EntitiesToMateVar);

// Set the Mate Alignment
MateData.MateAlignment = 1;

// Create the mate
MateFeature = (Feature)swAssembly.CreateMate(MateData);
Part.ClearSelection2(true);
Part.EditRebuild3();

boolstatus = Part.Extension.SelectByRay(-0.00247922125254263, 0.028363044378807, 0.043532545142341, -0.400036026779312,
boolstatus = Part.Extension.SelectByRay(0.114306836013213, 0.0719973430751679, -0.0518791456821077, -0.400036026779312,

// Create PerpendicularMateFeatureData
PerpMateData = (PerpendicularMateFeatureData)swAssembly.CreateMateData(2);

// Set the Entities To Mate
EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
EntitiesToMateVar = EntitiesToMate;
PerpMateData.EntitiesToMate = (EntitiesToMateVar);

// Create the mate
MateFeature = (Feature)swAssembly.CreateMate(PerpMateData);
Part.ClearSelection2(true);
Part.EditRebuild3();

boolstatus = Part.Extension.SelectByRay(-0.0130850967460105, 0.0540093074191645, 0.000810230676393076, -0.400036026779312,
boolstatus = Part.Extension.SelectByRay(0.107909556750883, 0.0719973430752248, -0.0658876969148992, -0.400036026779312,

// Create CoincidentMateFeatureData
CoincMateData = (CoincidentMateFeatureData)swAssembly.CreateMateData(0);

// Set the Entities To Mate
EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
EntitiesToMateVar = EntitiesToMate;
CoincMateData.EntitiesToMate = (EntitiesToMateVar);

// Set the Mate Alignment
CoincMateData.MateAlignment = 0;

// Create the mate
MateFeature = (Feature)swAssembly.CreateMate(CoincMateData);
Part.ClearSelection2(true);
Part.EditRebuild3();

boolstatus = Part.Extension.SelectByRay(-0.0265813760897231, 0.0540093074191077, -0.0191102240282248, -0.400036026779312,
boolstatus = Part.Extension.SelectByRay(0.104493667114582, 0.0540093074190509, -0.053508121263917, -0.400036026779312, -

// Create ParallelMateFeatureData
ParMateData = (ParallelMateFeatureData)swAssembly.CreateMateData(3);

// Set the Entities To Mate
EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
EntitiesToMateVar = EntitiesToMate;
ParMateData.EntitiesToMate = (EntitiesToMateVar);

// Set the Mate Alignment
ParMateData.MateAlignment = 0;

// Create the mate
MateFeature = (Feature)swAssembly.CreateMate(ParMateData);
Part.ClearSelection2(true);
Part.EditRebuild3();

boolstatus = Part.Extension.SelectByRay(0.153629139956536, 0.0267016961580566, -0.0207225117635517, -0.400036026779312,
boolstatus = Part.Extension.SelectByRay(0.31258643852118, 0.0771807121026882, -0.175728481540773, -0.400036026779312, -

// Create ConcentricMateFeatureData
ConcMateData = (ConcentricMateFeatureData)swAssembly.CreateMateData(1);

// Set the Entities To Mate
EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
EntitiesToMateVar = EntitiesToMate;
ConcMateData.EntitiesToMate = (EntitiesToMateVar);

// Set the Mate Alignment
ConcMateData.MateAlignment = 1;

// Set the Lock Rotation
ConcMateData.LockRotation = false;

// Create the mate
MateFeature = (Feature)swAssembly.CreateMate(ConcMateData);
Part.ClearSelection2(true);
Part.EditRebuild3();

```

```

        boolstatus = Part.Extension.SelectByID2("cylinder20-
1@assem20", "COMPONENT", 0.136504049029156, 0.042017793613752, -0.00604340129007142, true, 1, null, 0);
        boolstatus = Part.Extension.SelectByID2("shaft-
1@assem20", "COMPONENT", 0.116970722742224, 0.072421443211681, -0.0524867796554531, true, 1, null, 0);

        // Create LockMateFeatureData
        LMateData = (LockMateFeatureData)swAssembly.CreateMateData(16);

        // Set the Entities To Mate
        EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
        EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
        EntitiesToMateVar = EntitiesToMate;
        LMateData.EntitiesToMate = (EntitiesToMateVar);

        // Create the mate
        MateFeature = (Feature)swAssembly.CreateMate(LMateData);
        Part.ClearSelection2(true);
        Part.EditRebuild3();
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}

}

```

Create Standard Mates Example (VBA)

This example shows how to create a variety of standard mates.

```

'-----
' Preconditions: Ensure the specified file paths exist.
'
' Postconditions: Inspect the Mates folder in the FeatureManager design tree.
'
' NOTE: Because the models are used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim swAssembly As SldWorks.AssemblyDoc
Dim AssemblyTitle As String
Dim tmpObj As SldWorks.ModelDoc2
Dim swInsertedComponent As SldWorks.Component2
Dim errors As Long
Dim Part As SldWorks.ModelDoc2
Dim TransformData(15) As Double
Dim TransformDataVariant As Variant
Dim swMathUtil As Object
Dim swTransform As Object
Dim MateData As SldWorks.TangentMateFeatureData
Dim EntitiesToMate(1) As Object
Dim EntitiesToMateVar As Variant
Dim MateFeature As Feature
Dim PerpMateData As SldWorks.PerpendicularMateFeatureData
Dim CoincMateData As SldWorks.CoincidentMateFeatureData
Dim ParMateData As SldWorks.ParallelMateFeatureData
Dim ConcMateData As SldWorks.ConcentricMateFeatureData
Dim LMateData As SldWorks.LockMateFeatureData
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long

Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\assem20.sldasm", 2, 0, "",
longstatus, longwarnings)

    Set swAssembly = Part
    swApp.ActivateDoc2 "assem20.sldasm", False, longstatus
    Set Part = swApp.ActiveDoc

    AssemblyTitle = Part.GetTitle

    Set tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\shaft.sldprt", 1, 32,
"", errors, longwarnings)
    Set Part = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)

    Set swInsertedComponent = Part.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2019\samples\tutorial\api\shaft.sldprt", 0, "", False, "", 0.29642267129384, 9.20506109250709E-02, -0.187506963149644)
    swApp.CloseDoc "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\shaft.sldprt"

    TransformData(0) = 1
    TransformData(1) = 0
    TransformData(2) = 0
    TransformData(3) = 0
    TransformData(4) = 1
    TransformData(5) = 0
    TransformData(6) = 0
    TransformData(7) = 0
    TransformData(8) = 1
    TransformData(9) = 0.29642267129384
    TransformData(10) = 4.20506109250709E-02
    TransformData(11) = -0.187506963149644
    TransformData(12) = 1
    TransformData(13) = 0
    TransformData(14) = 0
    TransformData(15) = 0

    TransformDataVariant = TransformData

    Set swMathUtil = swApp.GetMathUtility()

    Set swTransform = swMathUtil.CreateTransform((TransformDataVariant))
    boolstatus = swInsertedComponent.SetTransformAndSolve2(swTransform)

    boolstatus = Part.Extension.SelectByRay(4.43527596829085E-02, 2.97941775455115E-02, 2.19712535516692E-03, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)
    boolstatus = Part.Extension.SelectByRay(0.130082347379926, 5.12737883206569E-02, -2.38688162796734E-02, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)

```

```

' Create TangentMateFeatureData
Set MateData = Part.CreateMateData(4)

' Set the Entities To Mate
Set EntitiesToMate(0) = Part.SelectionManager.GetSelectedObject6(1, -1)
Set EntitiesToMate(1) = Part.SelectionManager.GetSelectedObject6(2, -1)

EntitiesToMateVar = EntitiesToMate
MateData.EntitiesToMate = (EntitiesToMateVar)

' Set the Mate Alignment
MateData.MateAlignment = 1

' Create the mate
Set MateFeature = Part.CreateMate(MateData)
Part.ClearSelection2 True
Part.EditRebuild3

boolstatus = Part.Extension.SelectByRay(-2.47922125254263E-03, 0.028363044378807, 0.043532545142341, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)
boolstatus = Part.Extension.SelectByRay(0.114306836013213, 7.19973430751679E-02, -5.18791456821077E-02, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)

' Create PerpendicularMateFeatureData
Set PerpMateData = Part.CreateMateData(2)

' Set the Entities To Mate
Set EntitiesToMate(0) = Part.SelectionManager.GetSelectedObject6(1, -1)
Set EntitiesToMate(1) = Part.SelectionManager.GetSelectedObject6(2, -1)
EntitiesToMateVar = EntitiesToMate
PerpMateData.EntitiesToMate = (EntitiesToMateVar)

' Create the mate
Set MateFeature = Part.CreateMate(PerpMateData)
Part.ClearSelection2 True
Part.EditRebuild3

boolstatus = Part.Extension.SelectByRay(-1.30850967460105E-02, 5.40093074191645E-02, 8.10230676393076E-04, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)
boolstatus = Part.Extension.SelectByRay(0.107909556750883, 7.19973430752248E-02, -6.58876969148992E-02, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)

' Create CoincidentMateFeatureData
Set CoincMateData = Part.CreateMateData(0)

' Set the Entities To Mate
Set EntitiesToMate(0) = Part.SelectionManager.GetSelectedObject6(1, -1)
Set EntitiesToMate(1) = Part.SelectionManager.GetSelectedObject6(2, -1)
EntitiesToMateVar = EntitiesToMate
CoincMateData.EntitiesToMate = (EntitiesToMateVar)

' Set the Mate Alignment
CoincMateData.MateAlignment = 0

' Create the mate
Set MateFeature = Part.CreateMate(CoincMateData)
Part.ClearSelection2 True
Part.EditRebuild3

boolstatus = Part.Extension.SelectByRay(-2.65813760897231E-02, 5.40093074191077E-02, -1.91102240282248E-02, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)
boolstatus = Part.Extension.SelectByRay(0.104493667114582, 5.40093074190509E-02, -0.053508121263917, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)

' Create ParallelMateFeatureData
Set ParMateData = Part.CreateMateData(3)

' Set the Entities To Mate
Set EntitiesToMate(0) = Part.SelectionManager.GetSelectedObject6(1, -1)
Set EntitiesToMate(1) = Part.SelectionManager.GetSelectedObject6(2, -1)
EntitiesToMateVar = EntitiesToMate
ParMateData.EntitiesToMate = (EntitiesToMateVar)

' Set the Mate Alignment
ParMateData.MateAlignment = 0

' Create the mate
Set MateFeature = Part.CreateMate(ParMateData)
Part.ClearSelection2 True
Part.EditRebuild3

boolstatus = Part.Extension.SelectByRay(0.153629139956536, 2.67016961580566E-02, -2.07225117635517E-02, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)
boolstatus = Part.Extension.SelectByRay(0.31258643852118, 7.71807121026882E-02, -0.175728481540773, -0.400036026779312,

```

-0.515038074910024, -0.758094294050284, 2.99151229353486E-03, 2, True, 1, 0)

```
' Create ConcentricMateFeatureData
Set ConcMateData = Part.CreateMateData(1)
```

```
' Set the Entities To Mate
Set EntitiesToMate(0) = Part.SelectionManager.GetSelectedObject6(1, -1)
Set EntitiesToMate(1) = Part.SelectionManager.GetSelectedObject6(2, -1)
EntitiesToMateVar = EntitiesToMate
ConcMateData.EntitiesToMate = (EntitiesToMateVar)
```

```
' Set the Mate Alignment
ConcMateData.MateAlignment = 1
```

```
' Set the Lock Rotation
ConcMateData.LockRotation = False
```

```
' Create the mate
Set MateFeature = Part.CreateMate(ConcMateData)
Part.ClearSelection2 True
Part.EditRebuild3
```

```
boolstatus = Part.Extension.SelectByID2("cylinder20-1@assem20", "COMPONENT", 0.136504049029156, 0.042017793613752,
-6.04340129007142E-03, True, 1, Nothing, 0)
boolstatus = Part.Extension.SelectByID2("shaft-1@assem20", "COMPONENT", 0.116970722742224, 0.072421443211681,
-5.24867796554531E-02, True, 1, Nothing, 0)
```

```
' Create LockMateFeatureData
Set LMateData = Part.CreateMateData(16)
```

```
' Set the Entities To Mate
Set EntitiesToMate(0) = Part.SelectionManager.GetSelectedObject6(1, -1)
Set EntitiesToMate(1) = Part.SelectionManager.GetSelectedObject6(2, -1)
EntitiesToMateVar = EntitiesToMate
LMateData.EntitiesToMate = (EntitiesToMateVar)
```

```
' Create the mate
Set MateFeature = Part.CreateMate(LMateData)
Part.ClearSelection2 True
Part.EditRebuild3
```

End Sub

Create a Structural-Member Group Example (C#)

This example shows how to create structural-member groups for weldment features.

```
// -----
// Preconditions:
// 1. Ensure that the specified weldment profile and path exist.
// 2. If necessary, modify the path in both calls to InsertStructuralWeldment3.
//
// Postconditions:
// 1. Two structural-member features are created.
// 2. Each feature consists of one structural-member group of two
//    sketch segments.
// 3. Inspect the Immediate Window for information.
// -----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace CreateStructureMemberGroup_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        bool boolstatus;
        FeatureManager FeatMgr;
        SelectionMgr SelMgr;
        Feature swWeldFeat;
        StructuralMemberFeatureData swWeldFeatData;

        public void Main()
        {
            string MacroFolder = null;
            MacroFolder = swApp.GetCurrentMacroPathFolder();
            swApp.SetCurrentWorkingDirectory(MacroFolder);
            string Template = null;
            Template = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
            Part = (ModelDoc2)swApp.NewDocument(Template, 0, 0, 0);
            FeatMgr = Part.FeatureManager;
            SelMgr = (SelectionMgr)Part.SelectionManager;
            Part.ClearSelection2(true);
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.1872393706766, 0.1133237194389, 0, -0.07003610048208, 0.009188409684237, 0);
            Part.ClearSelection2(true);
            vSkLines = Part.SketchManager.CreateCornerRectangle(0.06513561531715, 0.03369083550887, 0, 0.1807053904567, -0.08106219210316, 0);
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            Part.ViewZoomtofit2();
            Feature myFeature = default(Feature);
            myFeature = Part.FeatureManager.InsertWeldmentFeature();
            StructuralMemberGroup Group1 = default(StructuralMemberGroup);
            Group1 = FeatMgr.CreateStructuralMemberGroup();
            SketchSegment[] segments1 = new SketchSegment[2];
            StructuralMemberGroup[] GroupArray1 = new StructuralMemberGroup[1];
            boolstatus = Part.Extension.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.1495427140733, 0.1133237194389, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.1872393706766, 0.08238014634844, 0, true, 0, null, 0);
            segments1[0] = (SketchSegment)SelMgr.GetSelectedObject6(1, 0);
            segments1[1] = (SketchSegment)SelMgr.GetSelectedObject6(2, 0);
            Group1.Segments = segments1;
            GroupArray1[0] = Group1;
            myFeature = Part.FeatureManager.InsertStructuralWeldment3("C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\weldment profiles\\ansi inch\\c channel\\3 x 5.sldlfp", 1, 0, false, GroupArray1);
            Part.ClearSelection2(true);
            StructuralMemberGroup Group2 = default(StructuralMemberGroup);
            Group2 = FeatMgr.CreateStructuralMemberGroup();
            SketchSegment[] segments2 = new SketchSegment[2];
            StructuralMemberGroup[] GroupArray2 = new StructuralMemberGroup[1];
            boolstatus = Part.Extension.SelectByID2("Line5@Sketch1", "EXTSKETCHSEGMENT", 0.1185825251083, 0.03369083550887, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line6@Sketch1", "EXTSKETCHSEGMENT", 0.06513561531715, -0.02774616865332, 0, true, 0, null, 0);
            segments2[0] = (SketchSegment)SelMgr.GetSelectedObject6(1, 0);
            segments2[1] = (SketchSegment)SelMgr.GetSelectedObject6(2, 0);
            Group2.Segments = segments2;
            GroupArray2[0] = Group2;
```

```

myFeature = Part.FeatureManager.InsertStructuralWeldment3("C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\weldment profiles\\ansi inch\\c channel\\3 x 5.sldlfp", 1, 0, false, GroupArray2);
Part.ClearSelection2(true);
// Get Group Information
StructuralMemberGroup Group = default(StructuralMemberGroup);
Object[] vGroups = null;
Object[] vSegments = null;
SketchSegment skSegment = default(SketchSegment);
boolstatus = Part.Extension.SelectByID2("Structural Member1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swWeldFeat = (Feature)SelMgr.GetSelectedObject6(1, 0);
swWeldFeatData = (StructuralMemberFeatureData)swWeldFeat.GetDefinition();
swWeldFeatData.AccessSelections(Part, null);
Debug.Print("");
Debug.Print("Groups Count : " + swWeldFeatData.GetGroupsCount());
Debug.Print(" Feature Name : " + swWeldFeat.Name);
vGroups = (Object[])swWeldFeatData.Groups;
long i = 0;
long j = 0;
for (i = vGroups.GetLowerBound(0); i <= vGroups.GetUpperBound(0); i++)
{
    Group = (StructuralMemberGroup)vGroups[i];
    Debug.Print(" Segment Count in Group " + i + 1 + " : " + Group.GetSegmentsCount());
    Debug.Print(" Rotational angle of group: " + Group.Angle.ToString());
    vSegments = (Object[])Group.Segments;
    for (j = vSegments.GetLowerBound(0); j <= vSegments.GetUpperBound(0); j++)
    {
        skSegment = (SketchSegment)vSegments[j];
        skSegment.Select(false);
    }
}
swWeldFeatData.ReleaseSelectionAccess();
boolstatus = Part.Extension.SelectByID2("Structural Member2", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swWeldFeat = (Feature)SelMgr.GetSelectedObject6(1, 0);
swWeldFeatData = (StructuralMemberFeatureData)swWeldFeat.GetDefinition();
swWeldFeatData.AccessSelections(Part, null);
Debug.Print("");
Debug.Print("Groups Count : " + swWeldFeatData.GetGroupsCount());
Debug.Print(" Feature Name : " + swWeldFeat.Name);
vGroups = (Object[])swWeldFeatData.Groups;
for (i = vGroups.GetLowerBound(0); i <= vGroups.GetUpperBound(0); i++)
{
    Group = (StructuralMemberGroup)vGroups[i];
    Debug.Print(" Segment Count in Group " + i + 1 + " : " + Group.GetSegmentsCount());
    Debug.Print(" Rotational angle of group: " + Group.Angle.ToString());
    vSegments = (Object[])Group.Segments;
    for (j = vSegments.GetLowerBound(0); j <= vSegments.GetUpperBound(0); j++)
    {
        skSegment = (SketchSegment)vSegments[j];
        skSegment.Select(false);
    }
}
swWeldFeatData.ReleaseSelectionAccess();
}
public SldWorks swApp;
}
}

```

Create a Structural-Member Group Example (VBA)

This example shows how to create structural-member groups for weldment features.

```

'-----
' Preconditions:
' 1. Ensure that the specified weldment profile and path exist.
' 2. If necessary, modify the path in both calls to
'   InsertStructuralWeldment3.
'
' Postconditions:
' 1. Two structural-member features are created.
' 2. Each feature consists of one structural-member group of two
'   sketch segments.
' 3. Inspect the Immediate Window for information.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As ModelDoc2
Dim boolstatus As Boolean
Dim FeatMgr As FeatureManager
Dim SelMgr As SelectionMgr
Dim swWeldFeat As SldWorks.Feature
Dim swWeldFeatData As SldWorks.StructuralMemberFeatureData
Option Explicit
Public Sub Main()
    Set swApp = Application.SldWorks

    Dim MacroFolder As String
    MacroFolder = swApp.GetCurrentMacroPathFolder()
    swApp.SetCurrentWorkingDirectory MacroFolder

    Dim Template As String
    Template = swApp.GetUserPreferenceStringValue(swDefaultTemplatePart)
    Set Part = swApp.NewDocument(Template, 0, 0, 0)
    Set FeatMgr = Part.FeatureManager
    Set SelMgr = Part.SelectionManager

    Part.ClearSelection2 True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.1872393706766, 0.1133237194389, 0, -0.07003610048208, 0.009188409684237, 0)
    Part.ClearSelection2 True
    vSkLines = Part.SketchManager.CreateCornerRectangle(0.06513561531715, 0.03369083550887, 0, 0.1807053904567, -0.08106219210316, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True

    Part.ViewZoomtofit2

    Dim myFeature As Object
    Set myFeature = Part.FeatureManager.InsertWeldmentFeature()

    Dim Group1 As StructuralMemberGroup
    Set Group1 = FeatMgr.CreateStructuralMemberGroup
    Dim vSeg1 As Variant
    Dim segments1(1) As Object
    Dim vGroups1 As Variant
    Dim GroupArray1(0) As Object
    boolstatus = Part.Extension.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.1495427140733, 0.1133237194389, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.1872393706766, 0.08238014634844, 0, True, 0, Nothing, 0)

    Set segments1(0) = SelMgr.GetSelectedObject6(1, 0)
    Set segments1(1) = SelMgr.GetSelectedObject6(2, 0)
    vSeg1 = segments1
    Group1.Segments = (vSeg1)
    Set GroupArray1(0) = Group1
    vGroups1 = GroupArray1

    Set myFeature = Part.FeatureManager.InsertStructuralWeldment3("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\c channel\3 x 5.sldflp", 1, 0, False, (vGroups1))
    Part.ClearSelection2 True

    Dim Group2 As StructuralMemberGroup
    Set Group2 = FeatMgr.CreateStructuralMemberGroup
    Dim vSeg2 As Variant

```

```

Dim segments2(1) As Object
Dim vGroups2 As Variant
Dim GroupArray2(0) As Object

boolstatus = Part.Extension.SelectByID2("Line5@Sketch1", "EXTSKETCHSEGMENT", 0.1185825251083, 0.03369083550887, 0, False, 0, Nothing, 0)
boolstatus = Part.Extension.SelectByID2("Line6@Sketch1", "EXTSKETCHSEGMENT", 0.06513561531715, -0.02774616865332, 0, True, 0, Nothing, 0)
Set segments2(0) = SelMgr.GetSelectedObject6(1, 0)
Set segments2(1) = SelMgr.GetSelectedObject6(2, 0)
vSeg2 = segments2
Group2.Segments = (vSeg2)
Set GroupArray2(0) = Group2
vGroups2 = GroupArray2

Set myFeature = Part.FeatureManager.InsertStructuralWeldment3("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\c channel\3 x 5.sldfp", 1, 0, False, (vGroups2))
Part.ClearSelection2 True

' Get Group Information

Dim Group As StructuralMemberGroup
Dim vGroups As Variant
Dim vSegments As Variant

boolstatus = Part.Extension.SelectByID2("Structural Member1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swWeldFeat = SelMgr.GetSelectedObject6(1, 0)

Set swWeldFeatData = swWeldFeat.GetDefinition
swWeldFeatData.AccessSelections Part, Nothing

Debug.Print ""
Debug.Print "Groups Count : " & swWeldFeatData.GroupsCount
Debug.Print " Feature Name : " & swWeldFeat.Name

vGroups = swWeldFeatData.Groups
Dim i As Long, j As Long
For i = LBound(vGroups) To UBound(vGroups)
    Set Group = vGroups(i)
    Debug.Print " Segment Count in Group " & i + 1 & " : " & Group.SegmentsCount; ""
    Debug.Print " Rotational angle of group: " & Group.Angle
    vSegments = Group.Segments
    For j = LBound(vSegments) To UBound(vSegments)
        vSegments(j).Select False
    Next j
Next i

swWeldFeatData.ReleaseSelectionAccess

boolstatus = Part.Extension.SelectByID2("Structural Member2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swWeldFeat = SelMgr.GetSelectedObject6(1, 0)

Set swWeldFeatData = swWeldFeat.GetDefinition
swWeldFeatData.AccessSelections Part, Nothing

Debug.Print ""
Debug.Print "Groups Count : " & swWeldFeatData.GroupsCount
Debug.Print " Feature Name : " & swWeldFeat.Name

vGroups = swWeldFeatData.Groups
For i = LBound(vGroups) To UBound(vGroups)
    Set Group = vGroups(i)
    Debug.Print " Segment Count in Group " & i + 1 & " : " & Group.SegmentsCount; ""
    Debug.Print " Rotational angle of group: " & Group.Angle
    vSegments = Group.Segments
    For j = LBound(vSegments) To UBound(vSegments)
        vSegments(j).Select False
    Next j
Next i

swWeldFeatData.ReleaseSelectionAccess
End Sub

```

Create a Structural-Member Group Example (VB.NET)

This example shows how to create structural-member groups for weldment features.

```

'-----
' Preconditions:
' 1. Ensure that the specified weldment profile and path exist.
' 2. If necessary, modify the path in both calls to
'   InsertStructuralWeldment3.
'
' Postconditions:
' 1. Two structural-member features are created.
' 2. Each feature consists of one structural-member group of two
'   sketch segments.
' 3. Inspect the Immediate Window for information.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Dim Part As ModelDoc2
    Dim boolstatus As Boolean
    Dim FeatMgr As FeatureManager
    Dim SelMgr As SelectionMgr
    Dim swWeldFeat As Feature
    Dim swWeldFeatData As StructuralMemberFeatureData

    Public Sub Main()
        Dim MacroFolder As String
        MacroFolder = swApp.GetCurrentMacroPathFolder()
        swApp.SetCurrentWorkingDirectory(MacroFolder)

        Dim Template As String
        Template = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
        Part = swApp.NewDocument(Template, 0, 0, 0)

        FeatMgr = Part.FeatureManager
        SelMgr = Part.SelectionManager
        Part.ClearSelection2(True)

        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.1872393706766, 0.1133237194389, 0, -0.07003610048208, 0.009188409684237, 0)
        Part.ClearSelection2(True)

        vSkLines = Part.SketchManager.CreateCornerRectangle(0.06513561531715, 0.03369083550887, 0, 0.1807053904567, -0.08106219210316, 0)
        Part.ClearSelection2(True)

        Part.SketchManager.InsertSketch(True)
        Part.ViewZoomtofit2()

        Dim myFeature As Feature
        myFeature = Part.FeatureManager.InsertWeldmentFeature()

        Dim Group1 As StructuralMemberGroup
        Group1 = FeatMgr.CreateStructuralMemberGroup
        Dim segments1(1) As SketchSegment

        Dim GroupArray1(0) As StructuralMemberGroup
        boolstatus = Part.Extension.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.1495427140733, 0.1133237194389, 0, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", -0.1872393706766, 0.08238014634844, 0, True, 0, Nothing, 0)
        segments1(0) = SelMgr.GetSelectedObject6(1, 0)
        segments1(1) = SelMgr.GetSelectedObject6(2, 0)
        Group1.Segments = segments1

        GroupArray1(0) = Group1
        myFeature = Part.FeatureManager.InsertStructuralWeldment3("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\c channel\3 x 5.sldlfp", 1, 0, False, GroupArray1)
        Part.ClearSelection2(True)

        Dim Group2 As StructuralMemberGroup
        Group2 = FeatMgr.CreateStructuralMemberGroup
        Dim segments2(1) As SketchSegment

        Dim GroupArray2(0) As StructuralMemberGroup
        boolstatus = Part.Extension.SelectByID2("Line5@Sketch1", "EXTSKETCHSEGMENT", 0.1185825251083, 0.03369083550887, 0, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line6@Sketch1", "EXTSKETCHSEGMENT", 0.06513561531715, -0.02774616865332, 0, True, 0, Nothing, 0)
        segments2(0) = SelMgr.GetSelectedObject6(1, 0)
        segments2(1) = SelMgr.GetSelectedObject6(2, 0)
        Group2.Segments = segments2

        GroupArray2(0) = Group2
        myFeature = Part.FeatureManager.InsertStructuralWeldment3("C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\weldment profiles\ansi inch\c channel\3 x 5.sldlfp", 1, 0, False, GroupArray2)
        Part.ClearSelection2(True)

        ' Get Group Information
    
```

```

Dim Group As StructuralMemberGroup
Dim vGroups As Object
Dim vSegments As Object
boolstatus = Part.Extension.SelectByID2("Structural Member1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swWeldFeat = SelMgr.GetSelectedObject6(1, 0)
swWeldFeatData = swWeldFeat.GetDefinition
swWeldFeatData.AccessSelections(Part, Nothing)
Debug.Print("")
Debug.Print("Groups Count : " & swWeldFeatData.GetGroupsCount)
Debug.Print(" Feature Name : " & swWeldFeat.Name)
vGroups = swWeldFeatData.Groups
Dim i As Long, j As Long
For i = LBound(vGroups) To UBound(vGroups)
    Group = vGroups(i)
    Debug.Print(" Segment Count in Group " & i + 1 & " : " & Group.GetSegmentsCount)
    Debug.Print(" Rotational angle for group: " + Group.Angle.ToString())
    vSegments = Group.Segments
    For j = LBound(vSegments) To UBound(vSegments)
        vSegments(j).Select(False)
    Next j
Next i
swWeldFeatData.ReleaseSelectionAccess()
boolstatus = Part.Extension.SelectByID2("Structural Member2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swWeldFeat = SelMgr.GetSelectedObject6(1, 0)
swWeldFeatData = swWeldFeat.GetDefinition
swWeldFeatData.AccessSelections(Part, Nothing)
Debug.Print("")
Debug.Print("Groups Count : " & swWeldFeatData.GetGroupsCount)
Debug.Print(" Feature Name : " & swWeldFeat.Name)
vGroups = swWeldFeatData.Groups
For i = LBound(vGroups) To UBound(vGroups)
    Group = vGroups(i)
    Debug.Print(" Segment Count in Group " & i + 1 & " : " & Group.GetSegmentsCount)
    Debug.Print(" Rotational angle for group: " + Group.Angle.ToString())
    vSegments = Group.Segments
    For j = LBound(vSegments) To UBound(vSegments)
        vSegments(j).Select(False)
    Next j
Next i
swWeldFeatData.ReleaseSelectionAccess()
End Sub

Public swApp As SldWorks
End Class

```

Create Submenus in the CommandManager Example (C#)

This example shows how to create submenus in the CommandManager framework by creating individual CommandGroups for each submenu in your tree. Create a C# addin using the SOLIDWORKS VSTA C# addin template. Set up CommandManager CommandGroups in SwAddin.ConnectToSW():

```
public bool ConnectToSW(SldWorks ThisSW, int Cookie)
{
    iApp = ThisSW;
    iCookie = Cookie;

    iApp.SetAddinCallbackInfo2(0, this, iCookie);

    iCommand = iApp.GetCommandManager(iCookie);

    SldWorks.CommandGroup commandGroup = null;
    string title = "CommandGroup1";
    string toolTip = "Command group 1 tip";
    string hint = "Command group 1 hint";
    int position = 6;
    commandGroup = iCommand.CreateCommandGroup(CommandGroupID1, title, toolTip, hint, position);
    if (commandGroup != null)
    {
        BitmapHandler bitmapHandler = new BitmapHandler();
        commandGroup.LargeIconList = bitmapHandler.CreateFileFromResource("CommandManager.LargeIconList.bmp");
        commandGroup.LargeMainIcon = bitmapHandler.CreateFileFromResource("CommandManager.LargeMainIcon.bmp");
        commandGroup.SmallIconList = bitmapHandler.CreateFileFromResource("CommandManager.SmallIconList.bmp");
        commandGroup.SmallMainIcon = bitmapHandler.CreateFileFromResource("CommandManager.SmallMainIcon.bmp");
        commandGroup.ShowInDocumentType = (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypeNONE;// |
//         (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypePART |
//         (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypeASSEMBLY |
//         (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypeDRAWING;

        commandGroup.AddCommandItem2("Command1", -1, "Command1 hint", "Command1 tool tip", 0, "OnCommand1", "OnMenuUpdate", 0, 1);
        commandGroup.AddCommandItem2("Command2", -1, "Command2 hint", "Command2 tool tip", 1, "OnCommand2", "OnMenuUpdate", 0, 1);

        commandGroup.HasMenu = true;
        commandGroup.HasToolbar = true;

        commandGroup.Activate();
    }

    commandGroup = null;
    title = "CommandGroup1\\CommandSubGroup1";
    toolTip = "Command sub group 1 tip";
    hint = "Command sub group 1 hint";
    position = -1;
    commandGroup = iCommand.CreateCommandGroup(CommandGroupID2, title, toolTip, hint, position);
    if (commandGroup != null)
    {
        BitmapHandler bitmapHandler = new BitmapHandler();
        commandGroup.LargeIconList = bitmapHandler.CreateFileFromResource("CommandManager.LargeIconList.bmp");
        commandGroup.LargeMainIcon = bitmapHandler.CreateFileFromResource("CommandManager.LargeMainIcon.bmp");
        commandGroup.SmallIconList = bitmapHandler.CreateFileFromResource("CommandManager.SmallIconList.bmp");
        commandGroup.SmallMainIcon = bitmapHandler.CreateFileFromResource("CommandManager.SmallMainIcon.bmp");
        commandGroup.ShowInDocumentType = (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypeNONE;// |
//         (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypePART |
//         (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypeASSEMBLY |
//         (int)SwConst.swDocTemplateTypes_e.swDocTemplateTypeDRAWING;

        commandGroup.AddCommandItem2("SubCommand1", -1, "Command1 hint", "Command1 tool tip", 0, "OnCommand1", "OnMenuUpdate", 0, 1);
        commandGroup.AddCommandItem2("SubCommand2", -1, "Command2 hint", "Command2 tool tip", 1, "OnCommand2", "OnMenuUpdate", 0, 1);

        commandGroup.HasMenu = true;
        commandGroup.HasToolbar = true;

        commandGroup.Activate();
    }
}
```

return true;

}

Create Surface-sweep Feature Example (C#)

This example shows how to create a surface-sweep feature.

```
//-----
// Preconditions: Verify that the part template exists.
//
// Postconditions:
// 1. Opens a new part.
// 2. Creates two sketches.
// 3. Inserts a surface-sweep feature.
// 4. Examine the FeatureManager design tree and
//    graphics area.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureManager = default(FeatureManager);
            bool status = false;

            //Open new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\templates\\part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create a sketch
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.0, 0.0, 0.068491, 0.049604, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.068491, 0.049604, 0.0, 0.10923, 0.112837, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.10923, 0.112837, 0.0, 0.194652, 0.154023, 0.0);
            swSketchManager.InsertSketch(true);

            swModel.ViewZoomtofit2();
            swModel.ShowNamedView2("Isometric", 7);
            swModel.ClearSelection2(true);

            //Create another sketch
            status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.0, 0.0, 0.0, 0.021042, 0.091756, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.021042, 0.091756, 0.0, 0.098366, 0.085093, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.098366, 0.085093, 0.0, 0.143062, 0.122696, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Insert surface sweep
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 4, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.InsertSweepSurface3(false, 0, false, false, 0, 0, 0, true, true, 0,
            true, false, 0, 0);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Create Surface-sweep Feature Example (VBA)

This example shows how to create a surface-sweep feature.

```

'-----
' Preconditions: Verify that the part template exists.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates two sketches.
' 3. Inserts a surface-sweep feature.
' 4. Examine the FeatureManager design tree and
'    graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swFeatureManager As SldWorks.FeatureManager
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    'Open new part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Create a sketch
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0#, 0#, 0.068491, 0.049604, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.068491, 0.049604, 0#, 0.10923, 0.112837, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.10923, 0.112837, 0#, 0.194652, 0.154023, 0#)
    swSketchManager.InsertSketch True

    swModel.ViewZoomtofit2
    swModel.ShowNamedView2 "Isometric", 7
    swModel.ClearSelection2 True

    'Create another sketch
    status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateLine(-0#, 0#, 0#, 0.021042, 0.091756, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.021042, 0.091756, 0#, 0.098366, 0.085093, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.098366, 0.085093, 0#, 0.143062, 0.122696, 0#)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Insert surface sweep
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 4, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.InsertSweepSurface3(False, 0, False, False, 0, 0, 0, True, True, 0, True, False, 0, 0)

End Sub

```

Create Surface-sweep Feature Example (VB.NET)

This example shows how to create a surface-sweep feature.

```

-----
' Preconditions: Verify that the part template exists.
-----
' Postconditions:
' 1. Opens a new part.
' 2. Creates two sketches.
' 3. Inserts a surface-sweep feature.
' 4. Examine the FeatureManager design tree and
'    graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swFeatureManager As FeatureManager
        Dim status As Boolean

        'Open new part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension

        'Create a sketch
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.0#, 0.0#, 0.068491, 0.049604, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.068491, 0.049604, 0.0#, 0.10923, 0.112837, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.10923, 0.112837, 0.0#, 0.194652, 0.154023, 0.0#)
        swSketchManager.InsertSketch(True)

        swModel.ViewZoomtofit2()
        swModel.ShowNamedView2("Isometric", 7)
        swModel.ClearSelection2(True)

        'Create another sketch
        status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateLine(-0.0#, 0.0#, 0.0#, 0.021042, 0.091756, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.021042, 0.091756, 0.0#, 0.098366, 0.085093, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.098366, 0.085093, 0.0#, 0.143062, 0.122690, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        'Insert surface sweep
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 4, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.InsertSweepSurface3(False, 0, False, False, 0, 0, 0, True, True, 0, True, False, 0, 0)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Surface Knit Feature Example (C#)

This example shows how to create a surface knit feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part template
// exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates two surfaces.
// 3. Creates a surface knit feature using the two
// selected surfaces.
// 4. Examine the graphics area, FeatureManager design
// tree, and Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SketchManager swSketchManager;
        SketchSegment swSketchSegment;
        FeatureManager swFeatureManager;
        SelectionMgr swSelectionManager;
        Feature swFeature;
        SurfaceKnitFeatureData swSurfaceKnitFeature;

        bool boolstatus;

        public void Main()
        {
            //Open new part document and create two surfaces
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prt", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(-0.0415374666666667, 0, 0, 0.0534585333333333, 0, 0, -0.0415374666666667, 0.029
86186666666667, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);
            swFeatureManager.FeatureExtrudeRefSurface2(true, false, false, 0, 0, 0.05, 0.01, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, false, false);
            swSelectionManager.EnableContourSelection = false;
            boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 1, null, 0);
            boolstatus = swModel.InsertPlanarRefSurface();
            swModel.ClearSelection2(true);

            // Select both surfaces and create surface knit feature
            boolstatus = swModelDocExt.SelectByID2("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, false, 1, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Surface-Plane1", "SURFACEBODY", 0, 0, 0, true, 1, null, 0);
            swFeature = (Feature)swFeatureManager.InsertSewRefSurface(true, false, false, 0.0001, 0.0001);

            // Get some surface knit feature data
            swSurfaceKnitFeature = (SurfaceKnitFeatureData)swFeature.GetDefinition();
            Debug.Print("Knit-surface feature: ");
            Debug.Print(" Knit tolerance: " + swSurfaceKnitFeature.KnitTolerance * 1000 + " mm");
            Debug.Print(" Maximum value for gap range: " + swSurfaceKnitFeature.MaxValueForGapRange * 1000 + " mm");
            Debug.Print(" Minimum value for gap range: " + swSurfaceKnitFeature.MinValueForGapRange * 1000 + " mm");
            Debug.Print(" Use gap filters? " + swSurfaceKnitFeature.UseGapFilters);
            Debug.Print(" Use merge entities? " + swSurfaceKnitFeature.UseMergeEntities);
            Debug.Print(" Try to form solid? " + swSurfaceKnitFeature.UseTryToFormSolid);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Surface Knit Feature Example (VBA)

This example shows how to create a surface knit feature and get its properties.

```

'-----
' Preconditions:
' 1. Verify that the specified part template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates two surfaces.
' 3. Creates a surface knit feature using the two
'    selected surfaces.
' 4. Examine the graphics area, FeatureManager design
'    tree, and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureManager As SldWorks.FeatureManager
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swSurfaceKnitFeature As SldWorks.SurfaceKnitFeatureData
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks

    'Open new part document and create two surfaces
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchManager = swModel.SketchManager
    Set swFeatureManager = swModel.FeatureManager
    Set swSelectionManager = swModel.SelectionManager
    boolstatus = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchManager.CreateEllipse(-4.1537466666667E-02, 0, 0, 5.3458533333333E-02, 0, 0, -4.1537466666667E-02, 2.0861866666667E-
02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    swFeatureManager.FeatureExtruRefSurface2 True, False, False, False, False, 0, 0, 0.05, 0.01, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02,
False, False, False, False, False, False, False, False, False
    swSelectionManager.EnableContourSelection = False
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)
    boolstatus = swModel.InsertPlanarRefSurface()
    swModel.ClearSelection2 True

    ' Select both surfaces and create surface knit feature
    boolstatus = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 1, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Surface-Plane1", "SURFACEBODY", 0, 0, 0, True, 1, Nothing, 0)
    Set swFeature = swFeatureManager.InsertSewRefSurface(True, False, False, 0.0001, 0.0001)

    ' Get some surface knit feature data
    Set swSurfaceKnitFeature = swFeature.GetDefinition
    Debug.Print "Knit-surface feature: "
    Debug.Print "  Knit tolerance: " & swSurfaceKnitFeature.KnitTolerance * 1000 & " mm"
    Debug.Print "  Maximum value for gap range: " & swSurfaceKnitFeature.MaxValueForGapRange * 1000 & " mm"
    Debug.Print "  Minimum value for gap range: " & swSurfaceKnitFeature.MinValueForGapRange * 1000 & " mm"
    Debug.Print "  Use gap filters? " & swSurfaceKnitFeature.UseGapFilters
    Debug.Print "  Use merge entities? " & swSurfaceKnitFeature.UseMergeEntities
    Debug.Print "  Try to form solid? " & swSurfaceKnitFeature.UseTryToFormSolid

End Sub

```

Create Surface Knit Feature Example (VB.NET)

This example shows how to create a surface knit feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates two surfaces.
' 3. Creates a surface knit feature using the two
'    selected surfaces.
' 4. Examine the graphics area, FeatureManager design
'    tree, and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSketchManager As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim swFeatureManager As FeatureManager
    Dim swSelectionManager As SelectionMgr
    Dim swFeature As Feature
    Dim swSurfaceKnitFeature As SurfaceKnitFeatureData
    Dim boolstatus As Boolean

    Public Sub main()

        'Open new part document and create two surfaces
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prt", 0, 0, 0)
        swModelDocExt = swModel.Extension
        swSketchManager = swModel.SketchManager
        swFeatureManager = swModel.FeatureManager
        swSelectionManager = swModel.SelectionManager
        boolstatus = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchManager.CreateEllipse(-0.0415374666666667, 0, 0, 0.0534585333333333, 0, 0, -0.0415374666666667, 0.0208618666666667, 0)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        swFeatureManager.FeatureExtrudeRefSurface2(True, False, False, 0, 0, 0.05, 0.01, False, False, False, False, 0.0174532925199433, 0.0174532925199433,
        False, False, False, False, False, False, False)
        swSelectionManager.EnableContourSelection = False
        boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)
        boolstatus = swModel.InsertPlanarRefSurface()
        swModel.ClearSelection2(True)

        ' Select both surfaces and create surface knit feature
        boolstatus = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 1, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID("Surface-Plane1", "SURFACEBODY", 0, 0, 0, True, 1, Nothing, 0)
        swFeature = swFeatureManager.InsertSewRefSurface(True, False, False, 0.0001, 0.0001)

        ' Get some surface knit feature data
        swSurfaceKnitFeature = swFeature.GetDefinition
        Debug.Print("Knit-surface feature: ")
        Debug.Print("  Knit tolerance: " & swSurfaceKnitFeature.KnitTolerance * 1000 & " mm")
        Debug.Print("  Maximum value for gap range: " & swSurfaceKnitFeature.MaxValueForGapRange * 1000 & " mm")
        Debug.Print("  Minimum value for gap range: " & swSurfaceKnitFeature.MinValueForGapRange * 1000 & " mm")
        Debug.Print("  Use gap filters? " & swSurfaceKnitFeature.UseGapFilters)
        Debug.Print("  Use merge entities? " & swSurfaceKnitFeature.UseMergeEntities)
        Debug.Print("  Try to form solid? " & swSurfaceKnitFeature.UseTryToFormSolid)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Swept Flange Example (C#)

This example shows how to create a swept flange on a sheet metal part.

```
//=====
// Preconditions: Ensure that the specified part template exists.
//
// Postconditions:
// 1. Base-Flange1 and Swept Flange1 are created.
// 2. Inspect the graphics area and the FeatureManager design tree.
// =====
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace CreateSweptFlange_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            Feature swFeat;
            Feature myFeature;
            FeatureManager swFeatMgr;
            SweptFlangeFeatureData swFeatData;
            CustomBendAllowance customBendAllowanceData;
            Feature swProfileSketch;
            SketchSegment skSegment;
            ModelDoc2 Part;
            PartDoc swPart;
            bool boolstatus;
            double swSheetWidth;
            swSheetWidth = 0;
            double swSheetHeight;
            swSheetHeight = 0;
            Part = (ModelDoc2)swApp.NewDocument(@"E:\Program Files\SOLIDWORKS
Corp\SOLIDWORKS\lang\english\Tutorial\part.prtdot", 0, swSheetWidth, swSheetHeight);

            swPart = (PartDoc)Part;
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Front", "PLANE", -0.0350345417518034, 0.019677523162802,
0.00511863136830445, false, 0, null, 0);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
(int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            boolstatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
(int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            object vSkLines;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0420403557341645, 0.0275066828701494, 0, 0.0475026757367474,
-0.0220443628675665, 0);
            Part.ClearSelection2(true);

            Part.ShowNamedView2("Trimetric", 8);
            Part.ViewZoomtofit2();

            customBendAllowanceData = Part.FeatureManager.CreateCustomBendAllowance();
            customBendAllowanceData.KFactor = 0.5;

            myFeature = Part.FeatureManager.InsertSheetMetalBaseFlange2(0.0007366, false, 0.0007366, 0.02, 0.01, false, 0, 0,
1, customBendAllowanceData, false, 0, 0.0001, 0.0001, 0.5, true, false, true, true);
```

```

Part.ClearSelection2(true);
Part.SketchManager.InsertSketch(true);
boolstatus = Part.Extension.SelectByID("", "FACE", 0.0441584745988735, 0.0275066828701256, -0.000252375262334681,
true, 0, null, 0);

skSegment = Part.SketchManager.CreateLine(0.047503, 0.0, 0.0, 0.047503, -0.015713, 0.0);
Part.ClearSelection2(true);
Part.SketchManager.InsertSketch(true);

boolstatus = Part.Extension.SelectByID2("Sketch6", "SKETCH", 0.0254585357375204, -0.00378791126417555,
-0.013876316631307, true, 0, null, 0);
boolstatus = Part.Extension.SelectByRay(0.0472949686339916, 0.0133307046879168, 0.000207707102561017,
-0.499999999999997, -0.707106781186554, -0.499999999999995, 0.000423592175091009, 1, true, 0, 0);
Part.ClearSelection2(true);

// Select the sketch for the profile
boolstatus = Part.Extension.SelectByID2("Sketch6", "SKETCH", 0.0254585357375204, -0.00378791126417555,
-0.013876316631307, true, 0, null, 0);
swProfileSketch = (Feature)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(1, -1);
Part.ClearSelection2(true);

// Select an edge for the path
Edge[] swPathObj = new Edge[1];
boolstatus = Part.Extension.SelectByRay(0.0472949686339916, 0.0133307046879168, 0.000207707102561017,
-0.499999999999997, -0.707106781186554, -0.499999999999995, 0.000423592175091009, 1, true, 0, 0);
swPathObj[0] = (Edge)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(1, -1);

Part.ClearSelection2(true);

swFeatMgr = Part.FeatureManager;

swFeatData = (SweptFlangeFeatureData)swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmSweptFlange);
swFeatData.EndOffset = 0;
swFeatData.FlangePosition = (int)swSweptFlangePositionTypes_e.swSweptFlangePositionType_MaterialInside;
swFeatData.FlattenAlongPath = false;
swFeatData.OverrideDefaultSheetMetalParameters = false;
swFeatData.Path = swPathObj;
swFeatData.Profile = swProfileSketch;
swFeatData.ReverseDirection = false;
swFeatData.StartOffset = 0;
swFeatData.TrimSideBends = false;
swFeatData.UseDefaultBendAllowance = true;
swFeatData.UseDefaultBendRelief = true;
swFeatData.UseDefaultRadius = false;
swFeatData.UseGaugeTable = false;
swFeatData.UseMaterialSheetMetalParameters = false;
swFeat = swFeatMgr.CreateFeature(swFeatData);
Part.ClearSelection2(true);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create Swept Flange Example (VB.NET)

This example shows how to create a swept flange on a sheet metal part.

```
=====
'Preconditions: Ensure that the specified part template exists.
```

```
,
```

```
'Postconditions:
```

```
'1. Base-Flange1 and Swept Flange1 are created.
```

```
'2. Inspect the graphics area and the FeatureManager design tree.
```

```
=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swFeat As Feature
```

```
        Dim myFeature As Feature
```

```
        Dim swFeatMgr As FeatureManager
```

```
        Dim swFeatData As SweptFlangeFeatureData
```

```
        Dim customBendAllowanceData As CustomBendAllowance
```

```
        Dim swProfileSketch As Feature
```

```
        Dim skSegment As SketchSegment
```

```
        Dim Part As ModelDoc2
```

```
        Dim swPart As PartDoc
```

```
        Dim boolstatus As Boolean
```

```
        Dim longstatus As Integer
```

```
        Dim swSheetWidth As Double
```

```
        swSheetWidth = 0
```

```
        Dim swSheetHeight As Double
```

```
        swSheetHeight = 0
```

```
        Part = swApp.NewDocument("E:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\Tutorial\part.prtdot", 0, swSheetWidth, swSheetHeight)
```

```
        swPart = Part
```

```
        swApp.ActivateDoc2("Part1", False, longstatus)
```

```
        Part = swApp.ActiveDoc
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        boolstatus = Part.Extension.SelectByID2("Front", "PLANE", -0.0350345417518034, 0.019677523162802, 0.00511863136830445, False, 0, Nothing, 0)
```

```
        Part.ClearSelection2(True)
```

```
        boolstatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
```

```
        boolstatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
```

```
        Dim vSkLines As Object
```

```
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0420403557341645, 0.0275066828701494, 0, 0.0475026757367474, -0.0220443628675665, 0)
```

```
        Part.ClearSelection2(True)
```

```
        Part.ShowNamedView2("**Trimetric", 8)
```

```
        Part.ViewZoomtofit2
```

```
        customBendAllowanceData = Part.FeatureManager.CreateCustomBendAllowance()
```

```
        customBendAllowanceData.KFactor = 0.5
```

```
        myFeature = Part.FeatureManager.InsertSheetMetalBaseFlange2(0.0007366, False, 0.0007366, 0.02, 0.01, False, 0, 0, 1, customBendAllowanceData, False, 0, 0.0001, 0.0001, 0.5, True, False, True, True)
```

```

Part.ClearSelection2(True)
Part.SketchManager.InsertSketch(True)
boolstatus = Part.Extension.SelectByID("", "FACE", 0.0441584745988735, 0.0275066828701256, -0.000252375262334681, True, 0,
Nothing, 0)

skSegment = Part.SketchManager.CreateLine(0.047503, 0#, 0#, 0.047503, -0.015713, 0#)
Part.ClearSelection2(True)
Part.SketchManager.InsertSketch(True)

boolstatus = Part.Extension.SelectByID("Sketch6", "SKETCH", 0.0254585357375204, -0.00378791126417555, -0.013876316631307,
True, 0, Nothing, 0)
boolstatus = Part.Extension.SelectByRay(0.0472949686339916, 0.0133307046879168, 0.000207707102561017, -0.499999999999997,
-0.707106781186554, -0.499999999999995, 0.000423592175091009, 1, True, 0, 0)
Part.ClearSelection2(True)

'Select the sketch for the profile
boolstatus = Part.Extension.SelectByID("Sketch6", "SKETCH", 0.0254585357375204, -0.00378791126417555, -0.013876316631307,
True, 0, Nothing, 0)
swProfileSketch = Part.SelectionManager.GetSelectedObject6(1, -1)
Part.ClearSelection2(True)

'Select an edge for the path
Dim swPathObj(0) As Edge
boolstatus = Part.Extension.SelectByRay(0.0472949686339916, 0.0133307046879168, 0.000207707102561017, -0.499999999999997,
-0.707106781186554, -0.499999999999995, 0.000423592175091009, 1, True, 0, 0)
swPathObj(0) = Part.SelectionManager.GetSelectedObject6(1, -1)

Part.ClearSelection2(True)

swFeatMgr = Part.FeatureManager

swFeatData = swFeatMgr.CreateDefinition(swFeatureNameID_e.swFmSweptFlange)
swFeatData.EndOffset = 0
swFeatData.FlangePosition = swSweptFlangePositionTypes_e.swSweptFlangePositionType_MaterialInside
swFeatData.FlattenAlongPath = False
swFeatData.OverrideDefaultSheetMetalParameters = False
swFeatData.Path = swPathObj
swFeatData.Profile = swProfileSketch
swFeatData.ReverseDirection = False
swFeatData.StartOffset = 0
swFeatData.TrimSideBends = False
swFeatData.UseDefaultBendAllowance = True
swFeatData.UseDefaultBendRelief = True
swFeatData.UseDefaultRadius = False
swFeatData.UseGaugeTable = False
swFeatData.UseMaterialSheetMetalParameters = False
swFeat = swFeatMgr.CreateFeature(swFeatData)
Part.ClearSelection2(True)

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Swept Flange Using Gauge Table Example (C#)

This example shows how to create a swept flange using a bend table on a non-sheet metal part.

```
// =====
// Preconditions:
// 1. Ensure that the paths to templates and gauge tables are valid.
// 2. Open an Immediate window.
// 3. Press F5 repeatedly and inspect the Immediate window and FeatureManager design tree as instructed.
//
// Postconditions:
// 1. Creates Sketch1 for the sweep path.
// 2. Creates Sketch2 for the sweep profile.
// 3. Creates Swept Flange1 using a gauge table installed with SOLIDWORKS.
// 4. Displays gauge table parameters.
// 5. Modifies Swept Flange1 to override the gauge number, bend radius, and gauge thickness.
// 6. Displays new gauge parameters.
// =====

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateSweptFlangeUsingGaugeTable_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            PartDoc swPart;
            ModelDoc2 swModel;
            Feature swFeat;
            FeatureManager swFeatMgr;
            SweptFlangeFeatureData swSweptFlangeFeatureData;
            SweptFlangeFeatureData featData;
            SheetMetalGaugeTableParameters smGaugeTableParam;
            SketchSegment skSegment;
            RefPlane myRefPlane;
            int swFeatNameID;
            Feature[] swSketch = new Feature[1];
            bool boolstatus;
            int errCode;
            double swSheetWidth;
            double swSheetHeight;
            string gaugePath = null;
            int gaugeCount;
            string[] gaugeNumbers;
```

```

int i;
int radiiCount;
double[] radii;
string newgaugePath = null;

swSheetWidth = 0;
swSheetHeight = 0;
swModel = (ModelDoc2)swApp.NewDocument(@"E:\Program Files\SOLIDWORKS
Corp\SOLIDWORKS\lang\english\Tutorial\part.prt", 0, swSheetWidth, swSheetHeight);

swPart = (PartDoc)swModel;
swModel = (ModelDoc2)swApp.ActiveDoc;

skSegment = swModel.SketchManager.Create3PointArc(-0.058601, -0.015313, 0.0, -0.003828, -0.021603, 0.0, 0.002265,
-0.05642, 0.0);
swModel.ClearSelection2(true);
swModel.SketchManager.InsertSketch(true); // Sketch1
swModel.ClearSelection2(true);

boolstatus = swModel.Extension.SelectByID2("Point2@Sketch1", "EXTSKETCHPOINT", -0.003828, -0.021603, 0, true, 0,
null, 0);
boolstatus = swModel.Extension.SelectByID2("Arc1@Sketch1", "EXTSKETCHSEGMENT", 0.00285760095397308,
-0.0342380271903227, 0, true, 1, null, 0);

myRefPlane = (RefPlane)swModel.FeatureManager.InsertRefPlane(4, 0, 2, 0, 0, 0);

boolstatus = swModel.Extension.SelectByID2("Plane1", "PLANE", 0.0103901931573406, -0.00917038599747196,
-0.00622490227027586, true, 0, null, 0);

swModel.SketchManager.InsertSketch(true);
skSegment = swModel.SketchManager.CreateLine(0.0, 0.0, 0.0, 0.0, 0.018316, 0.0);
skSegment = swModel.SketchManager.CreateLine(0.0, 0.018316, 0.0, 0.008362, 0.035435, 0.0);
swModel.ClearSelection2(true);
swModel.SketchManager.InsertSketch(true); // Sketch2

swFeatNameID = (int)swFeatureNameID_e.swFmSweptFlange;
swFeatMgr = swModel.FeatureManager;
swSweptFlangeFeatureData = (SweptFlangeFeatureData)swFeatMgr.CreateDefinition(swFeatNameID);

swModel.ClearSelection2(true);

// Select the sweep path
boolstatus = swModel.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
swSketch[0] = (Feature)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);

swSweptFlangeFeatureData.Path = swSketch;

// Select the sweep profile
boolstatus = swModel.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 0, null, 0);
swSweptFlangeFeatureData.Profile = (Feature)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);

errCode = swSweptFlangeFeatureData.GetErrorCodes();
Debug.Print("Swept flange definition error code: " + System.Convert.ToString(errCode));

System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for the swept flange definition error

swSweptFlangeFeatureData.UseMaterialSheetMetalParameters = false;
swSweptFlangeFeatureData.UseGaugeTable = true;

Debug.Print("Use gauge table? " + System.Convert.ToString(swSweptFlangeFeatureData.UseGaugeTable));

smGaugeTableParam = (SheetMetalGaugeTableParameters)swSweptFlangeFeatureData.GetGaugeTableParameters();
boolstatus = smGaugeTableParam.GetGaugeTablePath(gaugePath);

```

```

        if (boolstatus == false)
            smGaugeTableParam.SetGaugeTablePath(@"E:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\LANG\ENGLISH\SHEET METAL
            GAUGE TABLES\BEND ALLOWANCE MM SAMPLE.XLS");

        boolstatus = smGaugeTableParam.GetGaugeTablePath(gaugePath);
        Debug.Print("Got gauge table path? " + System.Convert.ToString(boolstatus));
        Debug.Print("Gauge table path: " + gaugePath);
        Debug.Print("Process: " + smGaugeTableParam.Process);

        gaugeCount = smGaugeTableParam.GetGaugeNumberCount();
        Debug.Print("Gauge number count: " + System.Convert.ToString(gaugeCount));

        Debug.Print("Available gauge numbers: ");

        gaugeNumbers = (string[])smGaugeTableParam.GetAvailableGaugeNumbers();

        for (i = 0; i <= gaugeCount - 1; i++)
            Debug.Print(gaugeNumbers[i]);
        Debug.Print("Current gauge number: " + smGaugeTableParam.GetCurrentGaugeNumber());

        System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for the current gauge number

        radiiCount = smGaugeTableParam.GetRadiiCount();
        Debug.Print("Bend radii count: " + System.Convert.ToString(smGaugeTableParam.GetRadiiCount()));

        Debug.Print("Available bend radii: ");
        radii = (double[])smGaugeTableParam.GetAvailableRadii();
        for (i = 0; i <= radiiCount - 1; i++)
            Debug.Print(radii[i].ToString());
        Debug.Print("Current bend radius: " + System.Convert.ToString(smGaugeTableParam.GetCurrentRadius()));
        Debug.Print("Thickness: " + System.Convert.ToString(smGaugeTableParam.GetThickness()));
        Debug.Print("Override thickness? " + System.Convert.ToString(smGaugeTableParam.OverrideThickness));
        Debug.Print("Override bend radius? " + System.Convert.ToString(smGaugeTableParam.OverrideRadius));
        Debug.Print("Reverse direction? " + System.Convert.ToString(smGaugeTableParam.ReverseDirection));

        System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for current bend radius and current gauge
thickness

        swSweptFlangeFeatureData.SetGaugeTableParameters(smGaugeTableParam);

        swFeat = swFeatMgr.CreateFeature(swSweptFlangeFeatureData);

        errCode = swSweptFlangeFeatureData.GetErrorCodes();
        Debug.Print("Swept flange creation error code: " + System.Convert.ToString(errCode));
        swModel.ClearSelection2(true);

        System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for the swept flange creation status
design tree
        // Observe Sheet-Metal, Swept Flange1, and Flat-Pattern in the FeatureManager

        // Set new gauge number and override gauge thickness and bend radius
        featData = (SweptFlangeFeatureData)swFeat.GetDefinition();
        smGaugeTableParam = (SheetMetalGaugeTableParameters)featData.GetGaugeTableParameters();

        smGaugeTableParam.ReverseDirection = true;
        smGaugeTableParam.SetCurrentGaugeNumber("Gauge 3");
        smGaugeTableParam.SetThickness(0.006, true);
        smGaugeTableParam.SetRadius(0.006, true);

        featData.SetGaugeTableParameters(smGaugeTableParam);
        boolstatus = swFeat.ModifyDefinition(featData, swModel, null)
        Debug.Print("Swept flange modification status: " + System.Convert.ToString(boolstatus));

```

```

        System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for the swept flange modification status

        // Get new gauge number
        boolstatus = featData.AccessSelections(swModel, null);
        smGaugeTableParam = (SheetMetalGaugeTableParameters)featData.GetGaugeTableParameters();

        boolstatus = smGaugeTableParam.GetGaugeTablePath(newgaugePath);
        Debug.Print("Got gauge table path? " + System.Convert.ToString(boolstatus));
        Debug.Print("Gauge table path: " + newgaugePath);

        Debug.Print("Process: " + smGaugeTableParam.Process);

        gaugeCount = smGaugeTableParam.GetGaugeNumberCount();
        Debug.Print("Gauge number count: " + System.Convert.ToString(gaugeCount));
        Debug.Print("Available gauge numbers: ");
        gaugeNumbers = (string[])smGaugeTableParam.GetAvailableGaugeNumbers();

        for (i = 0; i <= gaugeCount - 1; i++)
            Debug.Print(gaugeNumbers[i]);
        Debug.Print("Current gauge number: " + smGaugeTableParam.GetCurrentGaugeNumber());

        System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for the new gauge number

        // Get new bend radius and gauge thickness
        radiiCount = smGaugeTableParam.GetRadiiCount();
        Debug.Print("Bend radii count: " + System.Convert.ToString(radiiCount));

        Debug.Print("Available bend radii: ");
        radii = (double[])smGaugeTableParam.GetAvailableRadii();
        for (i = 0; i <= radiiCount - 1; i++)
            Debug.Print(radii[i].ToString());
        Debug.Print("Current bend radius: " + System.Convert.ToString(smGaugeTableParam.GetCurrentRadius()));

        Debug.Print("Thickness: " + System.Convert.ToString(smGaugeTableParam.GetThickness()));
        Debug.Print("Override thickness? " + System.Convert.ToString(smGaugeTableParam.OverrideThickness));
        Debug.Print("Override bend radius? " + System.Convert.ToString(smGaugeTableParam.OverrideRadius));
        Debug.Print("Reverse direction? " + System.Convert.ToString(smGaugeTableParam.ReverseDirection));

        featData.ReleaseSelectionAccess();

        System.Diagnostics.Debugger.Break(); // Inspect the Immediate window for the new bend radius and gauge thickness
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```